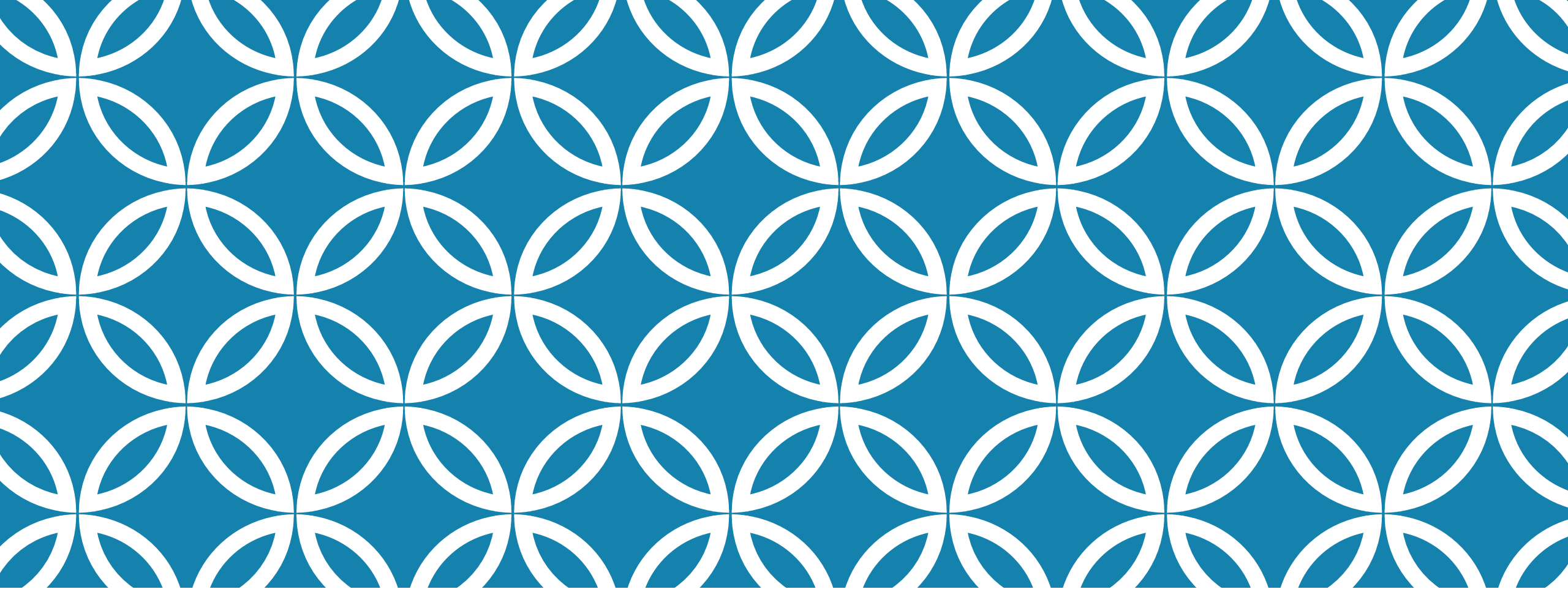




TÓPICOS EM PROCESSAMENTO PARALELO

Arquitetura de Computadores

PARALELISMO EM NÍVEL DE INSTRUÇÃO

Paralelismo em nível de instrução (*Instruction Level Parallelism – ILP*) requer 3 grandes tarefas

1. Checar dependências entre instruções para determinar quais instruções podem ser agrupadas para execução em paralelo

```
ADD AX, BX ; AX = AX + BX
```

```
INC BX      ; BX++
```

```
ADD AX, CX ; AX = AX + CX
```

2. Atribuir instruções às unidades funcionais (no hardware)

```
INC BX      → ULA Inteiros 1
```

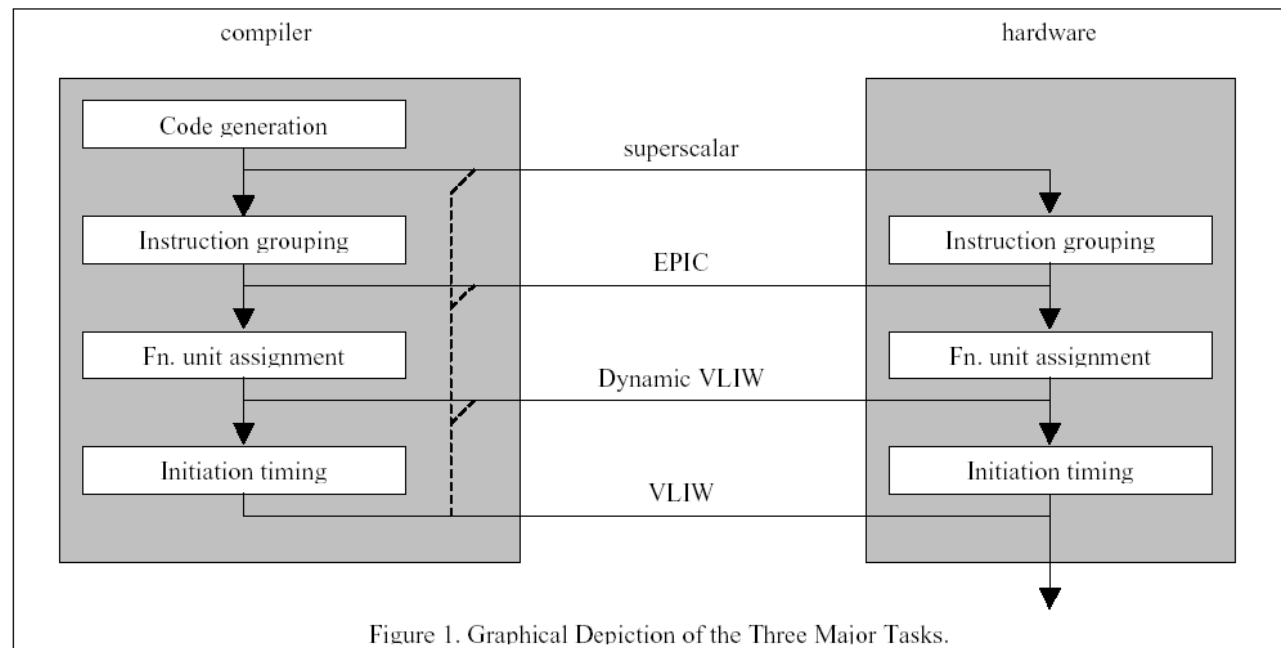
```
ADD AX, CX  → ULA Inteiros 2
```

3. Determinar quando instruções são iniciadas (i.e. são executadas)

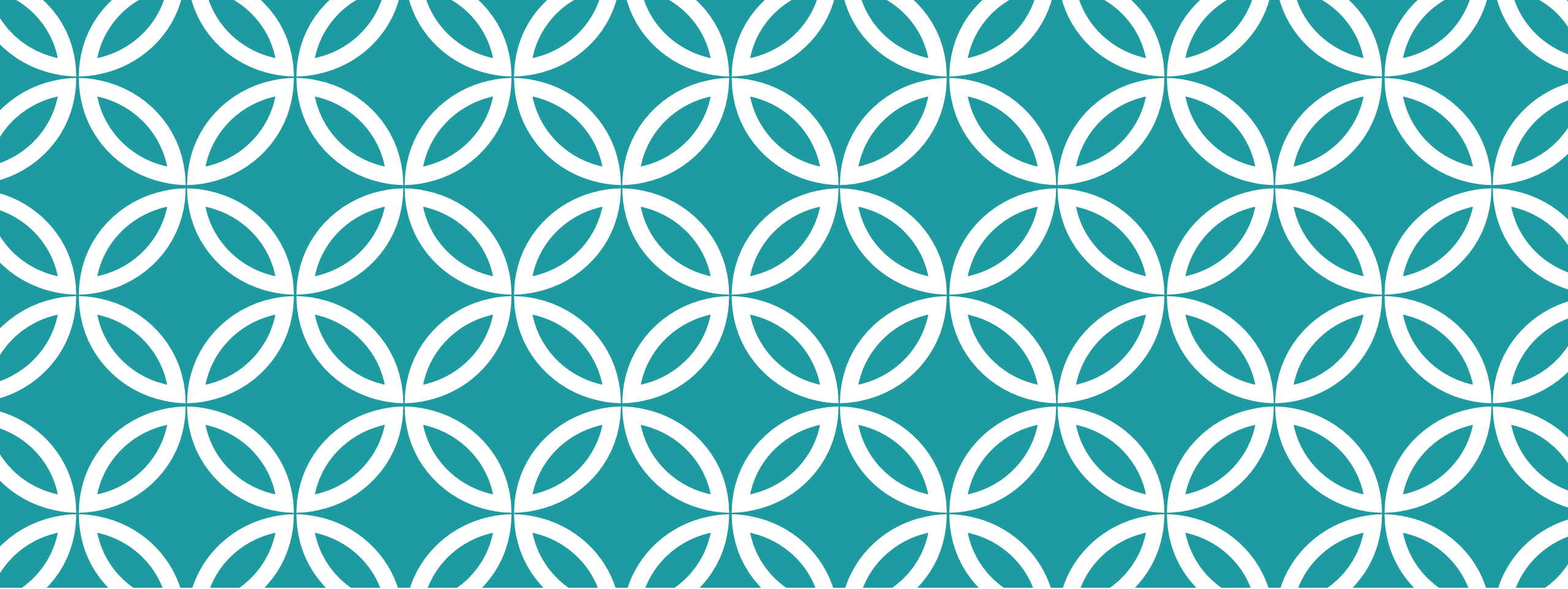
PARALELISMO EM NÍVEL DE INSTRUÇÃO: ARQUITETURAS

Diferentes maneiras de abordar o ILP

- Superscalar
- Very Long Instruction Word (VLIW)
- Explicitly Parallel Instruction Computing (EPIC)



Mark Smotherman, [“Understanding EPIC Architectures and Implementations”](#)



ARQUITETURAS SUPERESCALARES

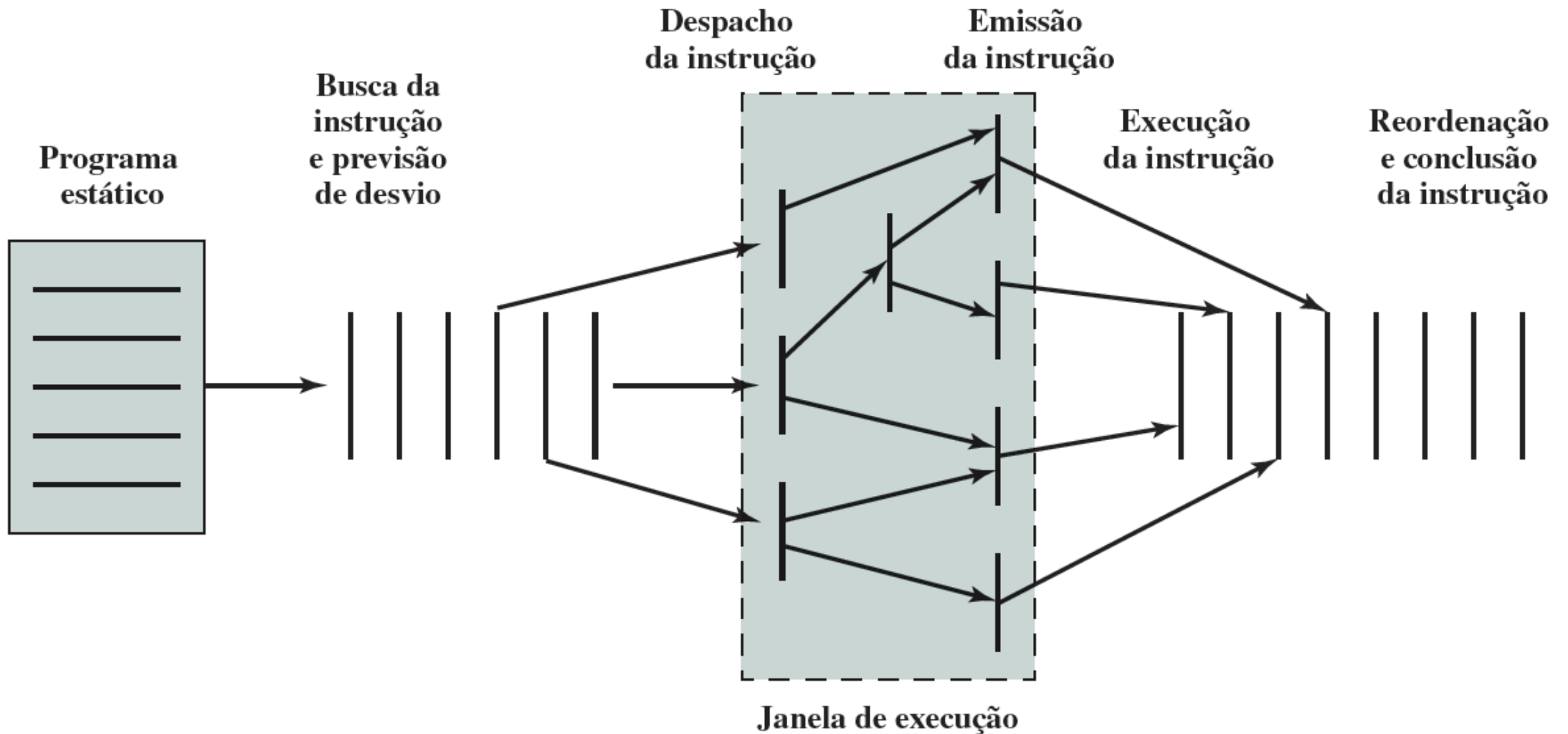
ARQUITETURAS SUPERESCALARES

Capazes de executar múltiplas instruções para cada ciclo de clock a partir de uma sequência linear de instruções por meio da **duplicação de unidades funcionais** (como ALUs, unidades de ponto flutuante e de acesso à memória, nas quais cada uma é implementada com um pipeline) e de mecanismos de **emissão e despacho paralelo** de instruções.

Todas as 3 tarefas (agrupamento, atribuição à unidades funcionais e a temporização) são realizadas pelo processador

- Tudo depende do tamanho da fila de instruções

ARQUITETURAS SUPERESCALARES



ARQUITETURAS SUPERESCALARES

Um processador superescalar pode executar código fora de ordem (*out of order*) fim de melhorar o desempenho da execução de instruções escalares

- Contém lógica em HW para determinar dependências verdadeiras envolvendo registradores

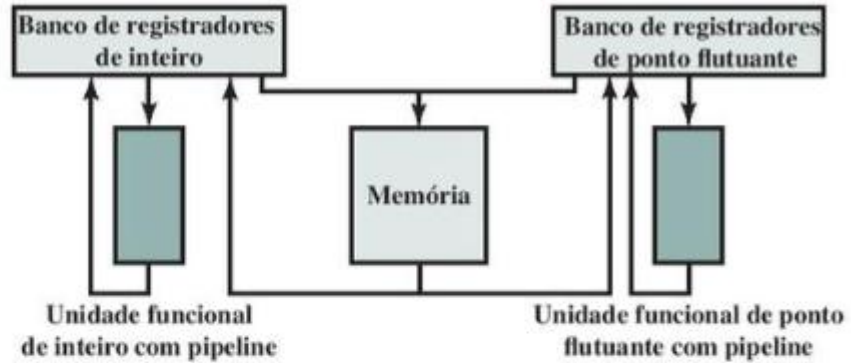
Instruções com durações diferentes introduzem latência na execução do programa

- A utilização do pipeline para estas instruções permite a execução de múltiplas instruções fora de ordem

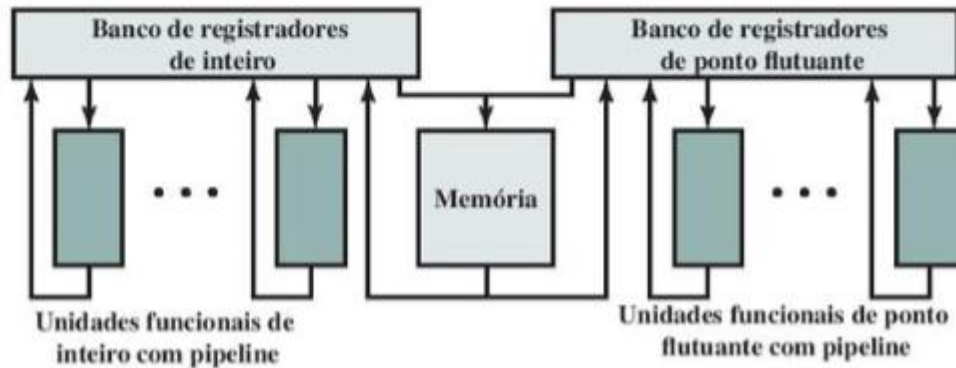
ARQUITETURAS SUPERESCALARES

Figura 16.1

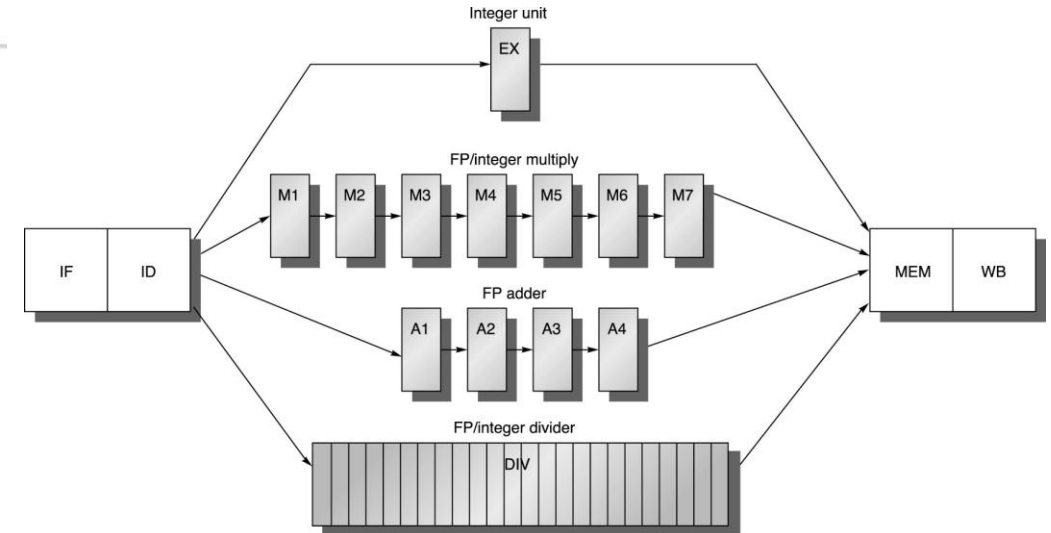
Organização superescalar comparada com organização escalar normal.



(a) Organização escalar

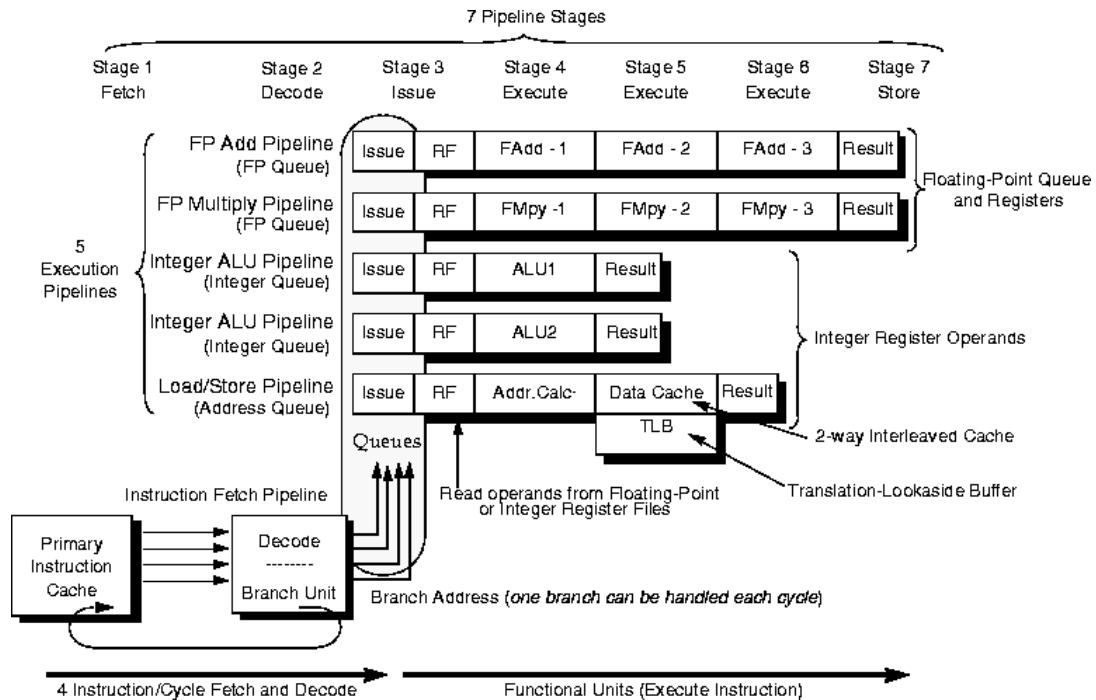


(b) Organização superescalar

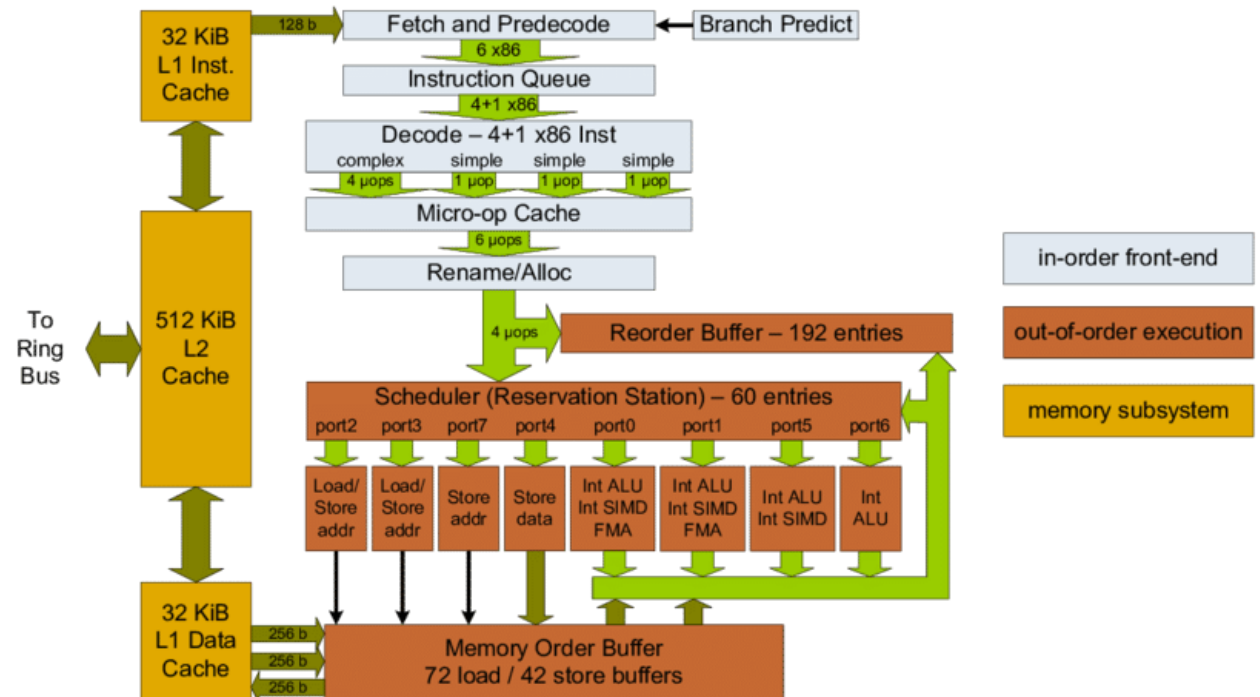


© 2003 Elsevier Science (USA). All rights reserved.

ARQUITETURAS SUPERESCALARES



MIPS R10000

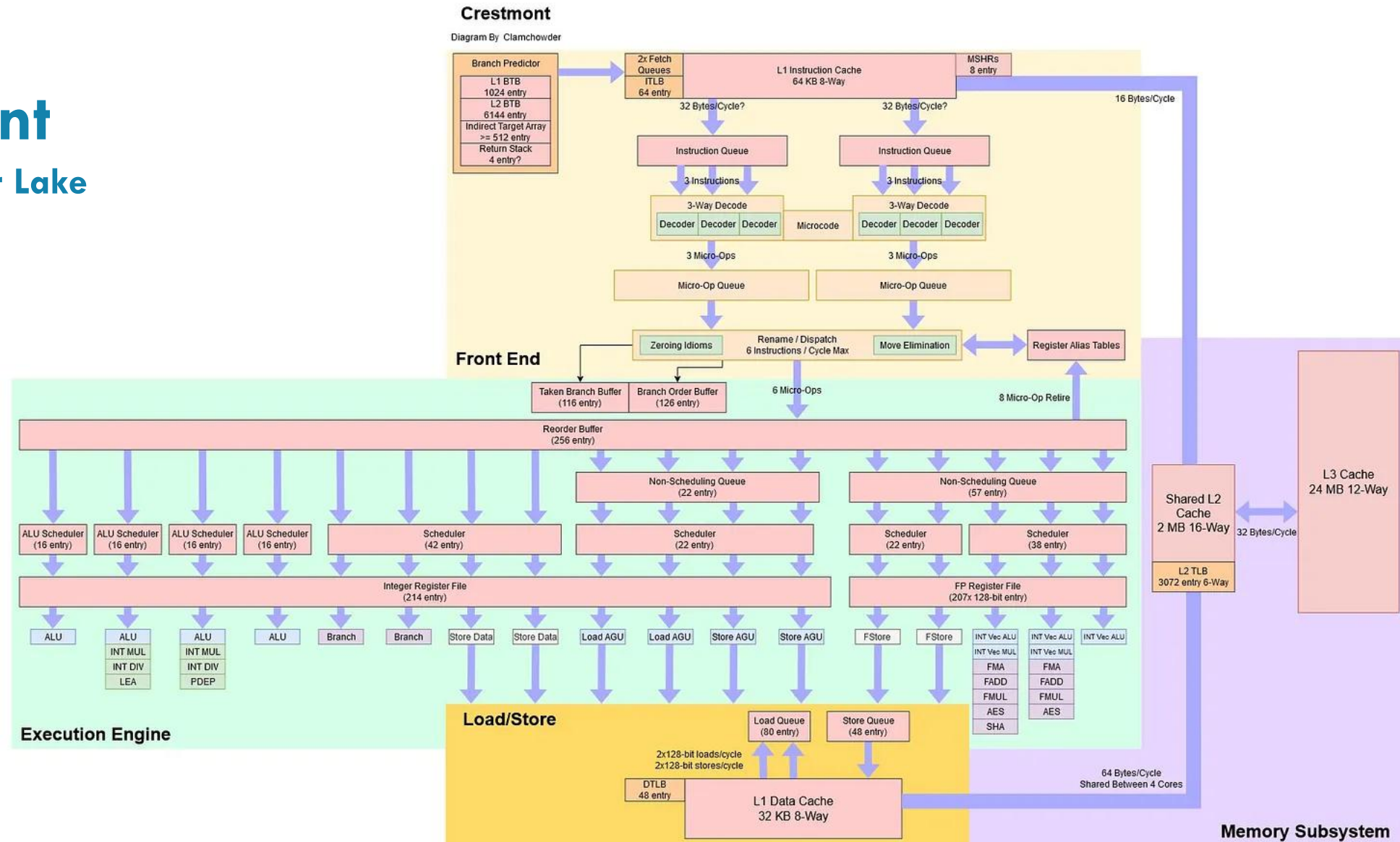


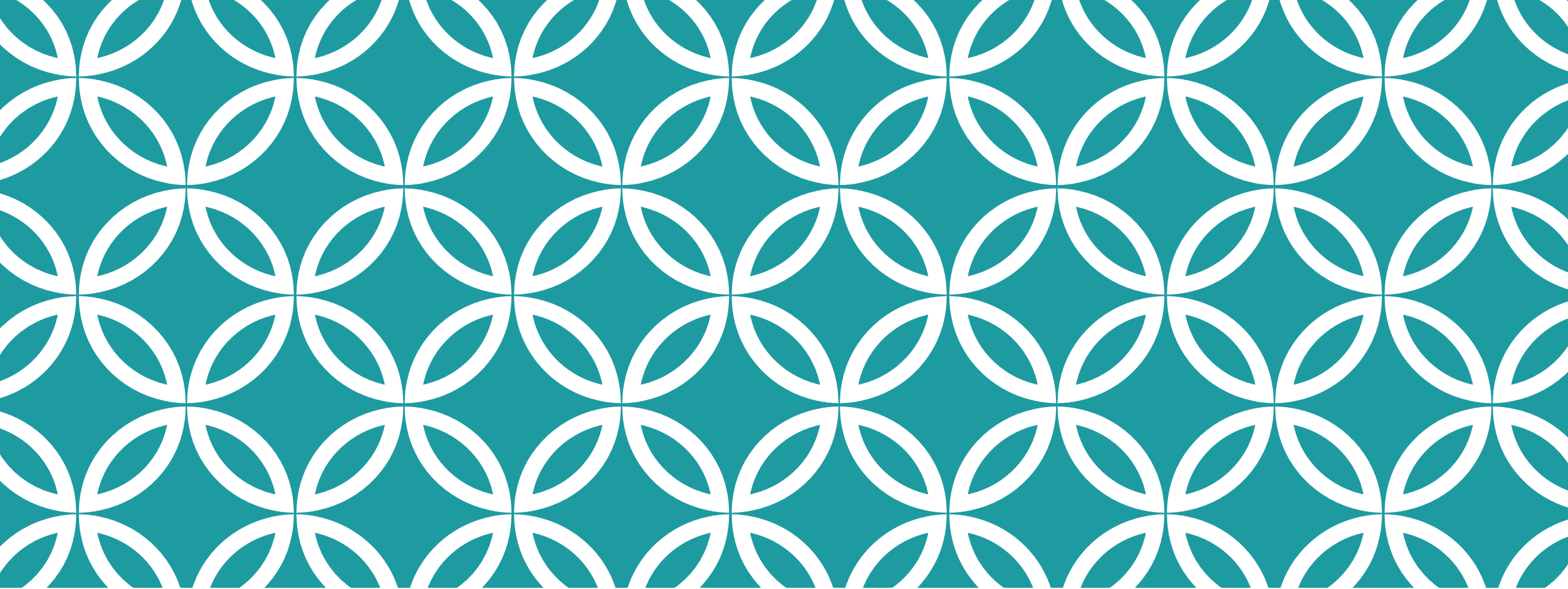
Intel Haswell

ARQUITETURAS SUPERESCALARES

Crestmont

GPU do Meteor Lake
(14ª Geração)



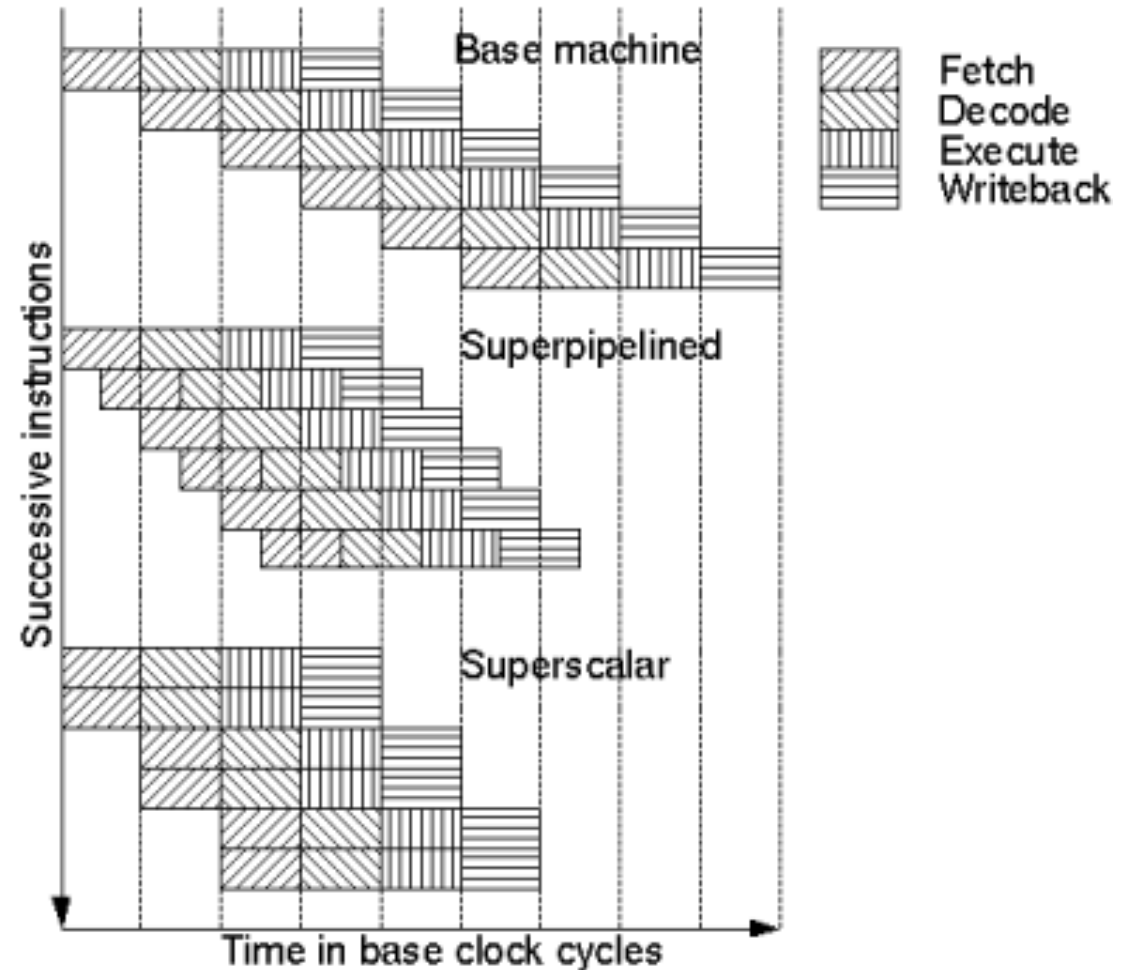


ARQUITETURAS SUPERPIPELINES

ARQUITETURAS SUPERPIPELINES

Uma arquitetura **superpipeline** é aquela em que o pipeline é subdividido em um número maior de estágios, permitindo que novas instruções sejam iniciadas em intervalos menores de tempo, com o objetivo de aumentar o paralelismo temporal e o **throughput (vazão)** do processador.

No superpipeline, cada um desses estágios pode ser dividido em dois (ou mais) subestágios — por exemplo, um pipeline de 10 ou 20 estágios.

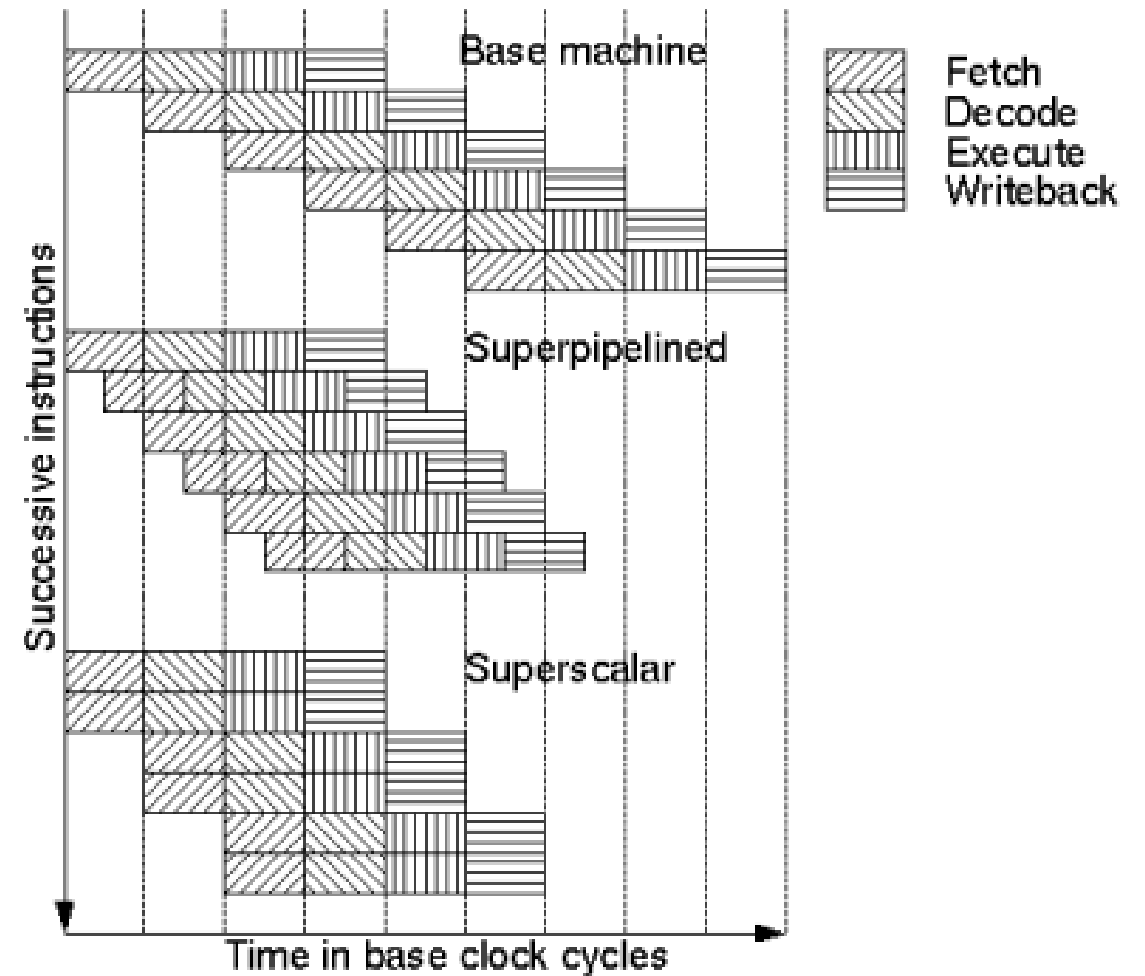


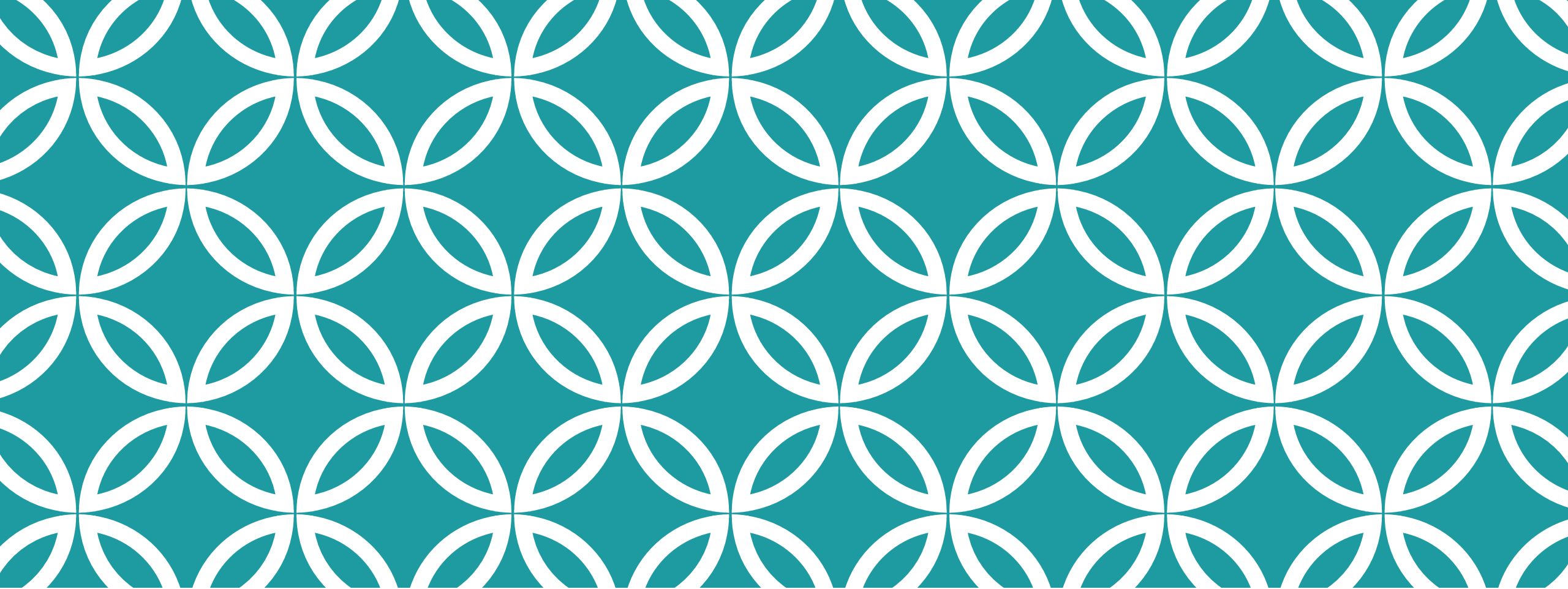
ARQUITETURAS SUPERPIPELINES

Superpipeline $\neq$ Superescalar

Desvantagens

- Maior complexidade de controle (por exemplo, detecção e tratamento de *hazards*).
- Maior penalidade em caso de desvios (*branch misprediction*).
- Dependências entre instruções tornam-se mais difíceis de resolver, exigindo técnicas como *forwarding* (o resultado de uma instrução diretamente da unidade funcional para outra instrução subsequente) e predição de desvios.





VERY LONG INSTRUCTION WORD

VLIW

VERY LONG INSTRUCTION WORD (VLIW)

Compilador descobre instruções que podem executar em paralelo

- Agrupa instruções formando uma longa instrução que será despachada para a máquina
- Máquinas VLIW têm tipicamente centenas de bits na instrução

$f3 = f4 \times f5$	$r6 = r7 + r8$	$r1 = \text{load}(r2)$	$\text{beq}(f4, f5)$
---------------------	----------------	------------------------	----------------------

MultiOp + UAL

- Decisões em tempo de compilação → Necessidade de compilador eficiente

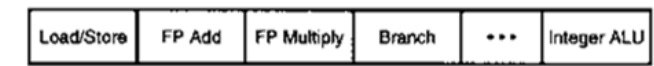
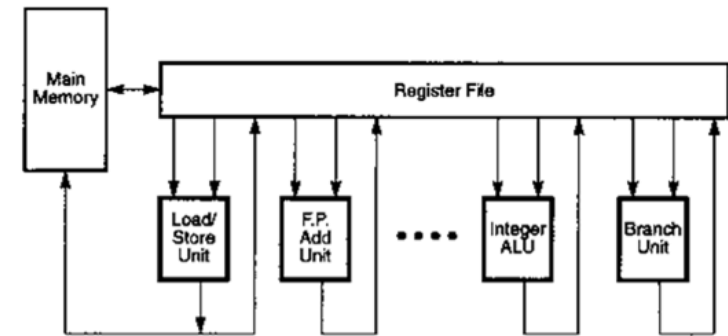
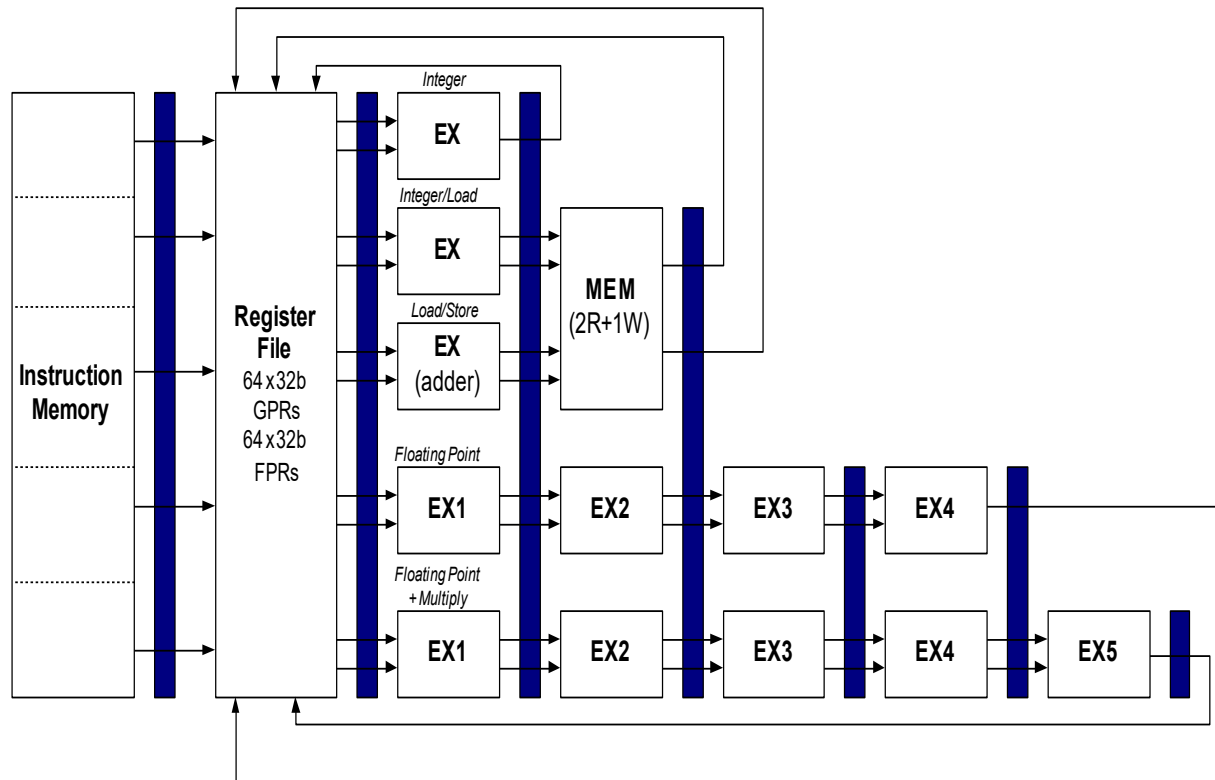
Não necessita verificar dependências em tempo de execução

Sem necessidade de escalonamento em tempo de execução

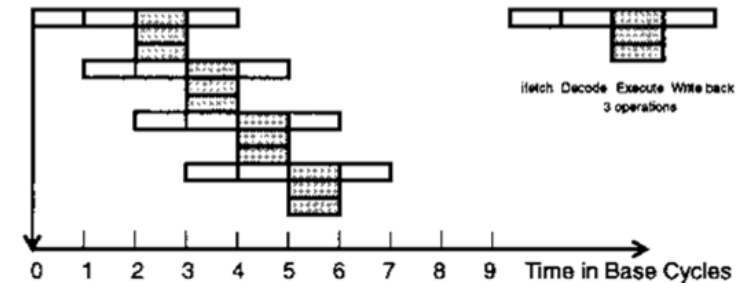
Múltiplas instruções contendo múltiplos fluxos de operação independentes

- Unidades funcionais executando instruções concorrentemente

VLIW: ARQUITETURA



(a) A typical VLIW processor and instruction format



(b) VLIW execution with degree $m = 3$

Figure 4.14 The architecture of a very long instruction word (VLIW) processor and its pipeline operations. (Courtesy of Multiflow Computer, Inc., 1987)

VLIW: VANTAGENS

Compilador prepara pacotes fixos de múltiplas operações que fornecem o “plano de execução”

- Dependências são determinadas pelo compilador e utilizadas para despachar as instruções de acordo com a latência da unidade funcional
- Unidades funcionais são atribuídas pelo compilador e correspondem a posição dentro do pacote de instruções ("slotting")
- Compilador produz código já definido para qual unidade funcional e sem dependência → HW não precisa “redescobrir” dependências ou definir que unidade funcional será utilizada

VLIW: DESVANTAGENS

Compatibilidade das implementações é um grande problema

- Código VLIW não rodará propriamente com diferente número de unidades funcionais ou com diferentes latências
- Eventos não previstos (por exemplo, cache miss) travará o processador
 - Dynamic VLIW delega para o HW quando a instrução deve iniciar permitindo lidar com eventos não previstos

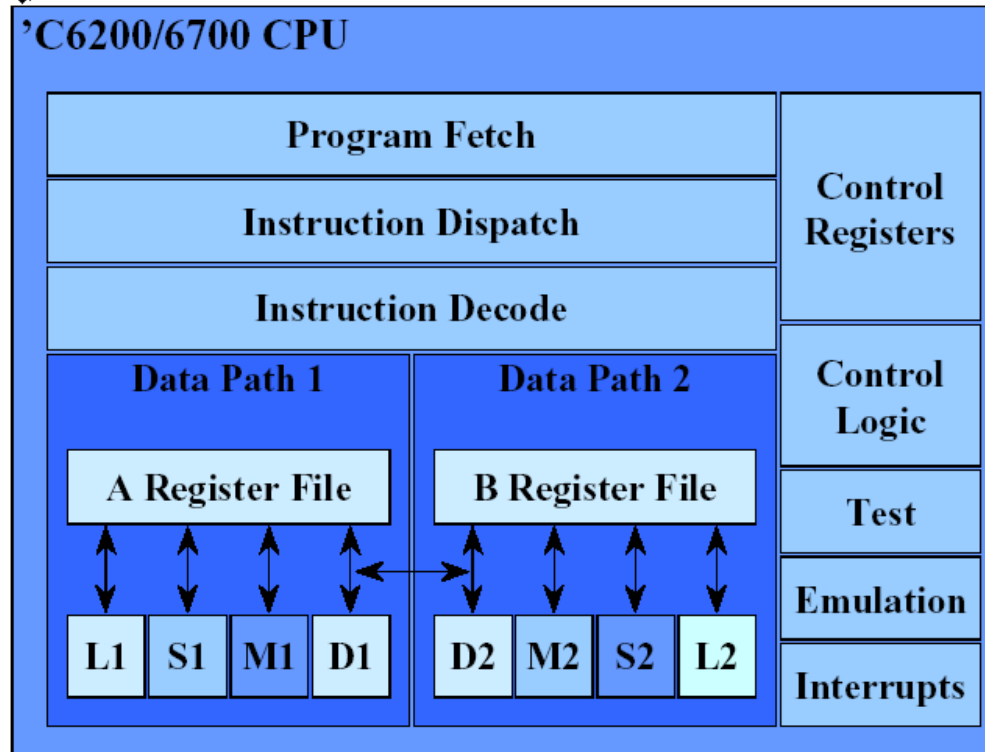
Densidade do código é outro problema

- Baixa utilização do slot (preenchidos com NOP)
- Reduz NOP por compressão ("VLIW flexível", "VLIW de tamanho variável")

VLIW: EXEMPLO

TMS320C6000 CPUs

18



◆ Advanced VLIW CPU (VelociTI™)

- Load-Store RISC
- Dual Identical Data Paths
 - 4 Functional Units /Each
 - Fetches 8 x 32-Bit Instructions/cycle
- 2 16 x 16 Integer Multipliers
 - 2 Multiply ACcumulates/cycle
 ↗ (MAC)
- 32/40-bit arithmetic
- Byte-Addressable

◆ 'C6200 Integer CPU

- 4 ns cycle time
- 2000 MIPS @ 250 MHz
- 500 MMACS (Mega MACs per Second)

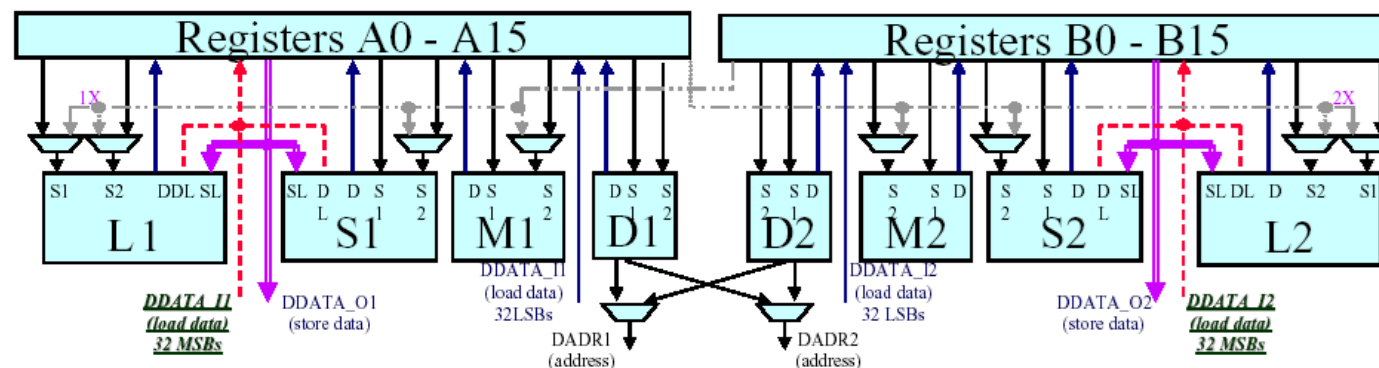
VLIW: EXEMPLO

Datapath Detail

30



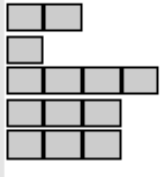
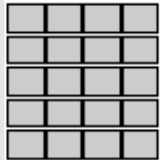
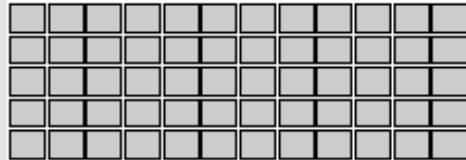
- ◆ 2 Data Paths
 - ◆ 8 Functional Units
 - Upto 6 Integer ALUs
 - 2 Integer/Floating Point Multipliers
 - 2 Floating Point Arithmetic Units
 - 2 Floating Point Auxiliary Units
 - ◆ Control
 - Independent
 - Up to 8 32-bit Instructions
 - ◆ Registers
 - 2 Files
 - 32, 32-bit registers total
 - ◆ Cross paths (1X, 2X)
- ◆ L-Unit (L1, L2)
 - Floating-Point ALU
 - 40-bit Integer ALU/Comparisons
 - Bit Counting
 - Normalization
 - ◆ S-Unit (S1, S2)
 - Floating Point Auxiliary Unit
 - 32-bit ALU/40-bit shifter
 - Bitfield Operations
 - Branching
 - ◆ M-Unit (M1, M2)
 - 16 x 16 to 32 Integer Multiplier
 - Floating-Point Multiplier
 - ◆ D-Unit (D1, D2)
 - 8-/16-/32-Bit Loads Stores
 - 64-bit loads
 - 32-bit add/subtract
 - Address Calculations
- Cross Paths
 - - - 40-bit Write Paths (8 MSBs)/64 Bit Load Path (32 MSBs)
 ⇒ 40-bit Read Paths/Store Paths
'C6700 Only'

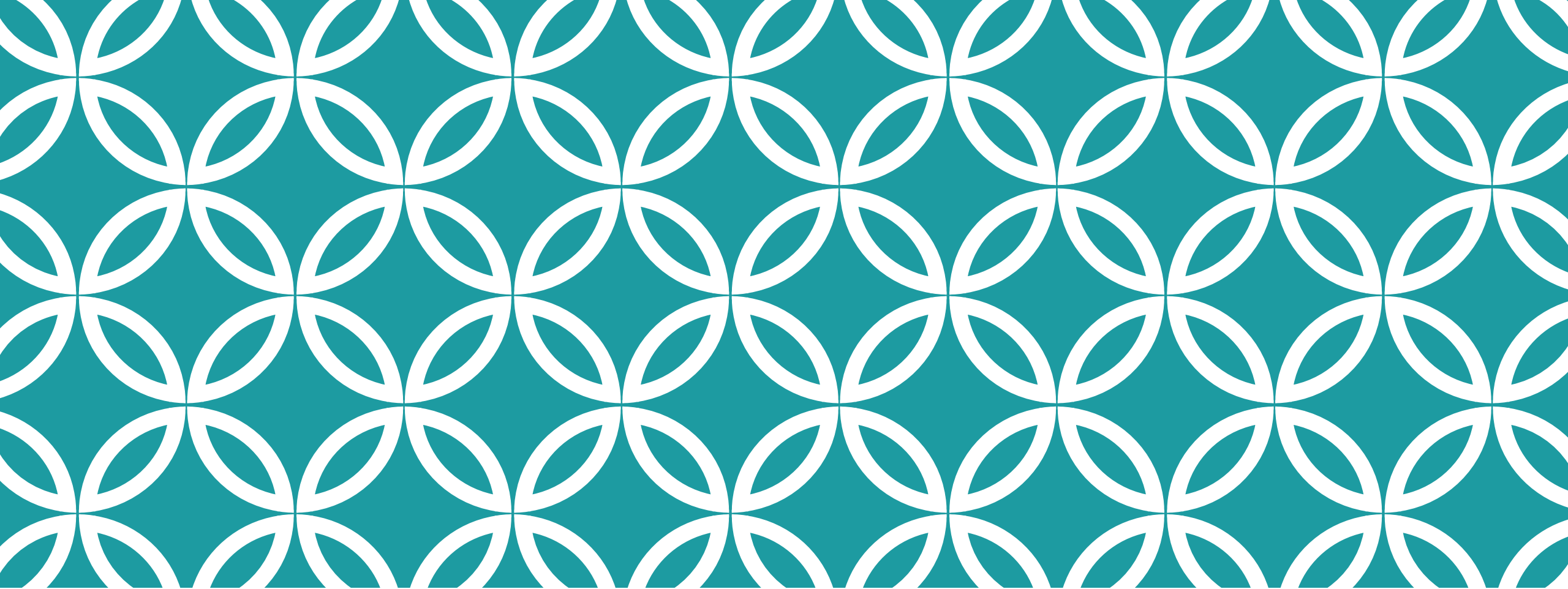


VLIW X SUPERESCALAR

Critério de comparação	Superescalar	VLIW
Detecção de paralelismo	Hardware	Compilador
Tempo disponível para realizar a detecção	Pouco	Muito
Conflito entre instruções	Possível	Não ocorre
Relógio (Clock)	Mais lento	Mais rápido
Tamanho de código	menor	maior
Compatibilidade através de gerações de HW	Sim	Não
Complexidade	Hardware	Software
Consumo de energia	maior	menor

CISC X RISC X VLIW

ARCHITECTURE CHARACTERISTIC	CISC	RISC	VLIW
INSTRUCTION SIZE	Varies	One size, usually 32 bits	One size
INSTRUCTION FORMAT	Field placement varies	Regular, consistent placement of fields	Regular, consistent placement of fields
INSTRUCTION SEMANTICS	Varies from simple to complex; possibly many dependent operations per instruction	Almost always one simple operation	Many simple, independent operations
REGISTERS	Few, sometimes special	Many, general-purpose	Many, general-purpose
MEMORY REFERENCES	Bundled with operations in many different types of instructions	Not bundled with operations, i.e., load/store architecture	Not bundled with operations, i.e., load/store architecture
HARDWARE DESIGN FOCUS	Exploit microcoded implementations	Exploit implementations with one pipeline and no microcode	Exploit implementations with multiple pipelines, no microcode & no complex dispatch logic
PICTURE OF FIVE TYPICAL INSTRUCTIONS  = 1 BYTE			

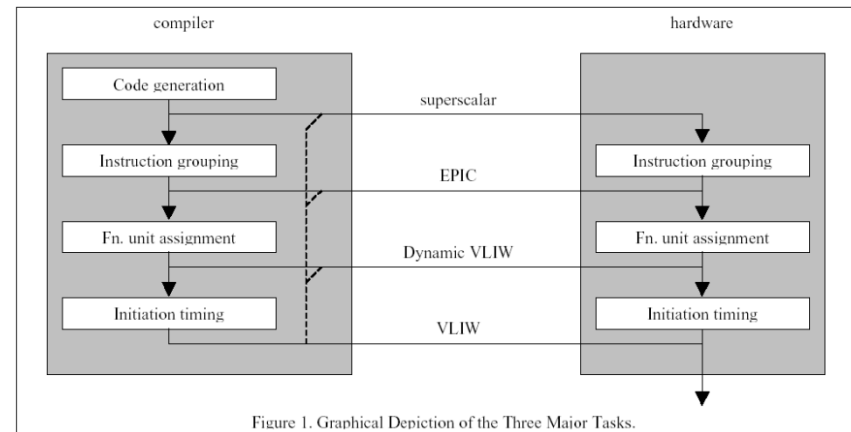


EXPLICITLY PARALLEL INSTRUCTION COMPUTING

EPIC

EPIC

Definida pela aliança Intel-HP, esta arquitetura implementa uma forma de arquitetura VLIW, na qual somente o agrupamento é realizado pelo compilador



A arquitetura EPIC é uma evolução da arquitetura VLIW

- Conjunto de instruções VLIW não são compatíveis entre implementações (diferentes unidades funcionais, diferentes pacotes de instruções)
- O escalonamento de instruções de carga de dados pode tornar-se muito complexo por conta do não determinismo da hierarquia de memória

EPIC

As palavras de instrução são pré-carregadas pelo HW, que identifica e depois escalona os pacotes em grupos independentes para execução paralela

Pacotes de instruções (chamados de “bundles”) podem ter tamanhos variáveis

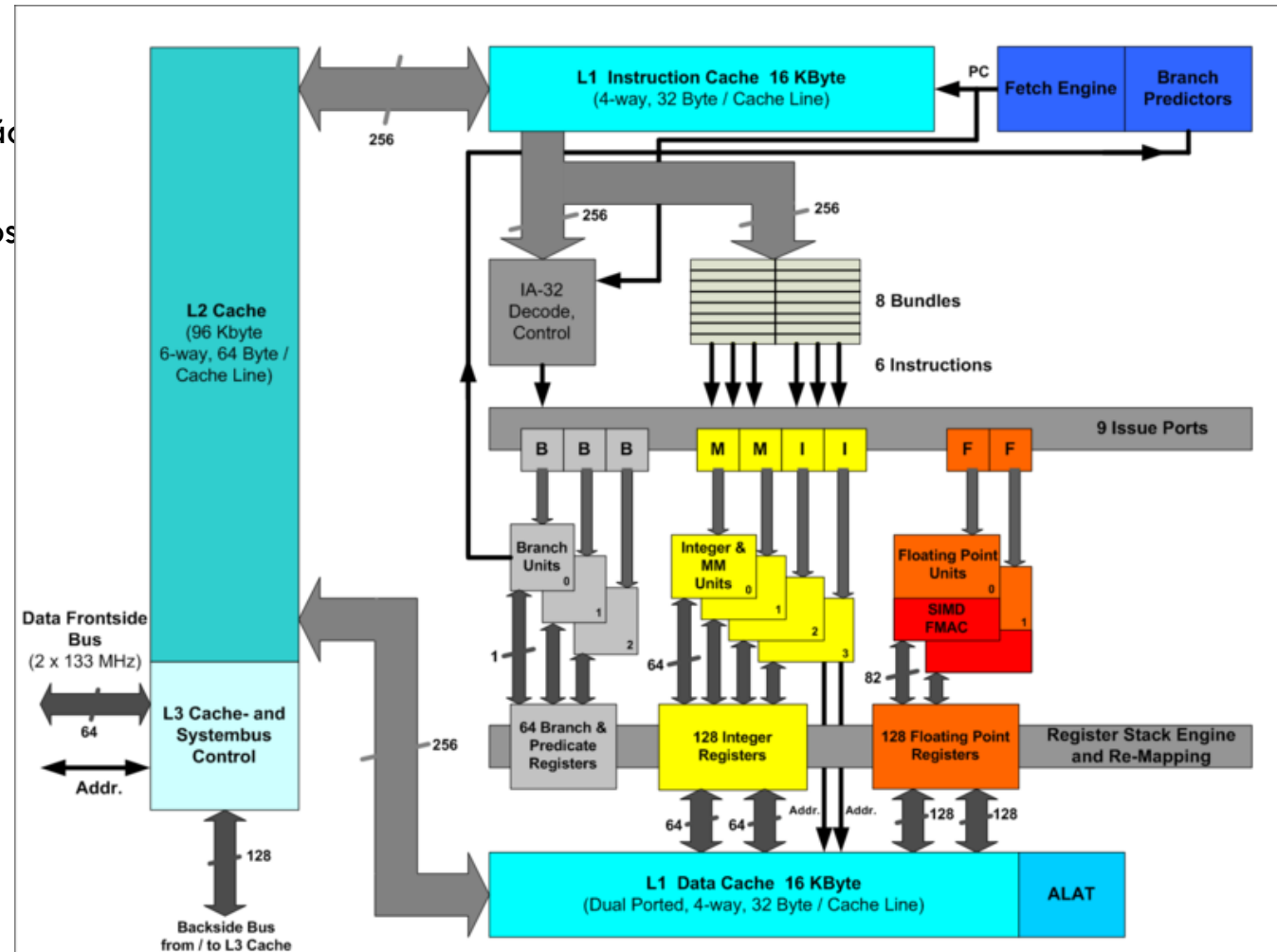
- *Variable-length Instruction Word – VLIW?*
- Delimitador especial demarca término e início de outro pacote

EPIC foi implementada na arquitetura Itanium (Intel IA-64), para servidores e sistemas de computação de alto desempenho

EPIC: INTEL IA-64

Registradores

- 128 GPRs (64 bits)
 - Os registradores de uso geral GPR0 a GPR31 são fixos
 - Os registradores 32-127 podem ser renomeados sob o controle do programa
- 128 FPR (82 bits)
- 8 registradores de desvio,
- 64 registradores de predicado (1 bit)
 - Os registradores de predicado servem para execução condicional das operações. Caso a execução da instrução não se confirme, o seu resultado é descartado
 - Os registradores de 16 a 63 podem ser renomeados em conjunto com os registradores de uso geral



EPIC: INTEL IA-64

Pacotes (bundles) de 128 bits com 3 instruções (41 bits) + predicado (5 bits)

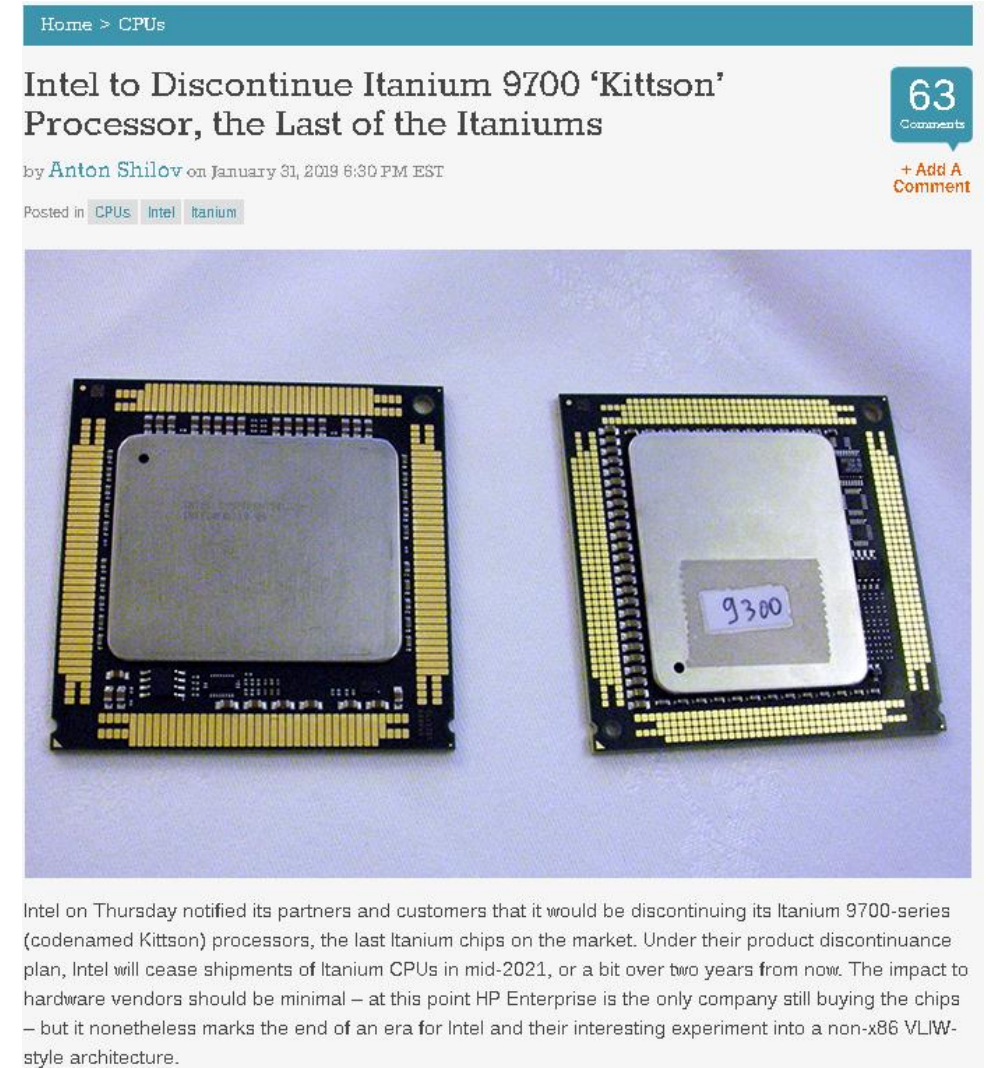
- Template IA-64 (predicado) especifica
 - **Tipo de operação para cada instrução:** define o tipo funcional de cada slot (posição) no pacote:
 - M – Acesso à memória (Memory)
 - I – Operação inteira (Integer)
 - F – Operação em ponto flutuante (Floating-point)
 - B – Desvio (Branch)
 - L – Operação de long latency (Long latency, como multiplicações)
 - Exemplo de tipos de template: MFI, MMI, MII, MLI, MIB, MMF, MFB, MMB, MBB, BBB
 - **Relacionamento intra-pacote:** indica se as instruções dentro do pacote podem ser executadas em paralelo ou se há dependências entre elas. Esse controle de paralelismo é **explícito**, ou seja, informado pelo compilador via *template*, não determinado dinamicamente pelo hardware (como nos processadores superescalares).
 - $M / MI \rightarrow$ a instrução “M” é independente e pode ser executada em paralelo com o grupo “MI”.
 - $MI / I \rightarrow$ as duas primeiras instruções “MI” formam um grupo com dependência, e a última “I” pode ser executada separadamente.
 - **Relacionamento inter-pacote:** indica se o próximo pacote (bundle) pode ser executado simultaneamente ao atual ou se deve esperar a conclusão dele.

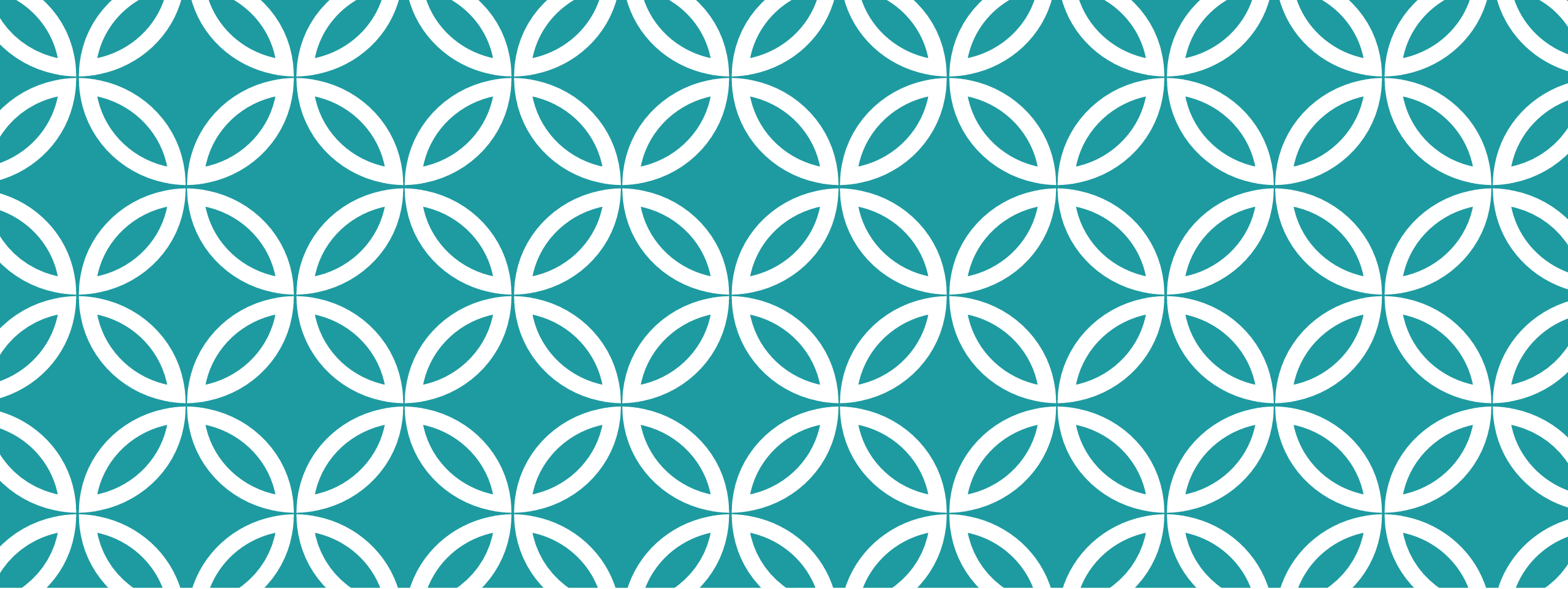
EPIC: AINDA EXISTE?

A Intel descontinuou o processador Itanium – Série 9700 ‘Kittson’ – em 2020

Algumas razões para o seu fim

1. **Baixa Adoção e Demanda Decrescente**
2. **Concorrência com Arquitetura x86-64**
3. **Falta de Suporte de Software**
4. **Custo Alto e Complexidade**
5. **Mudança de Foco da HP**





HYPER-THREADING

HT

HYPER-THREADING

Abordagem multi-thread

- Duas ou mais threads podem executar virtualmente de forma simultânea no mesmo processador
- Arquiteturas multi-threads são conhecidas como SMT (Simultaneous Multi-Threading)

Hyper-Threading é um caso de SMT proposto pela Intel

- Disponível em processadores Xeon, Pentium 4, Atom

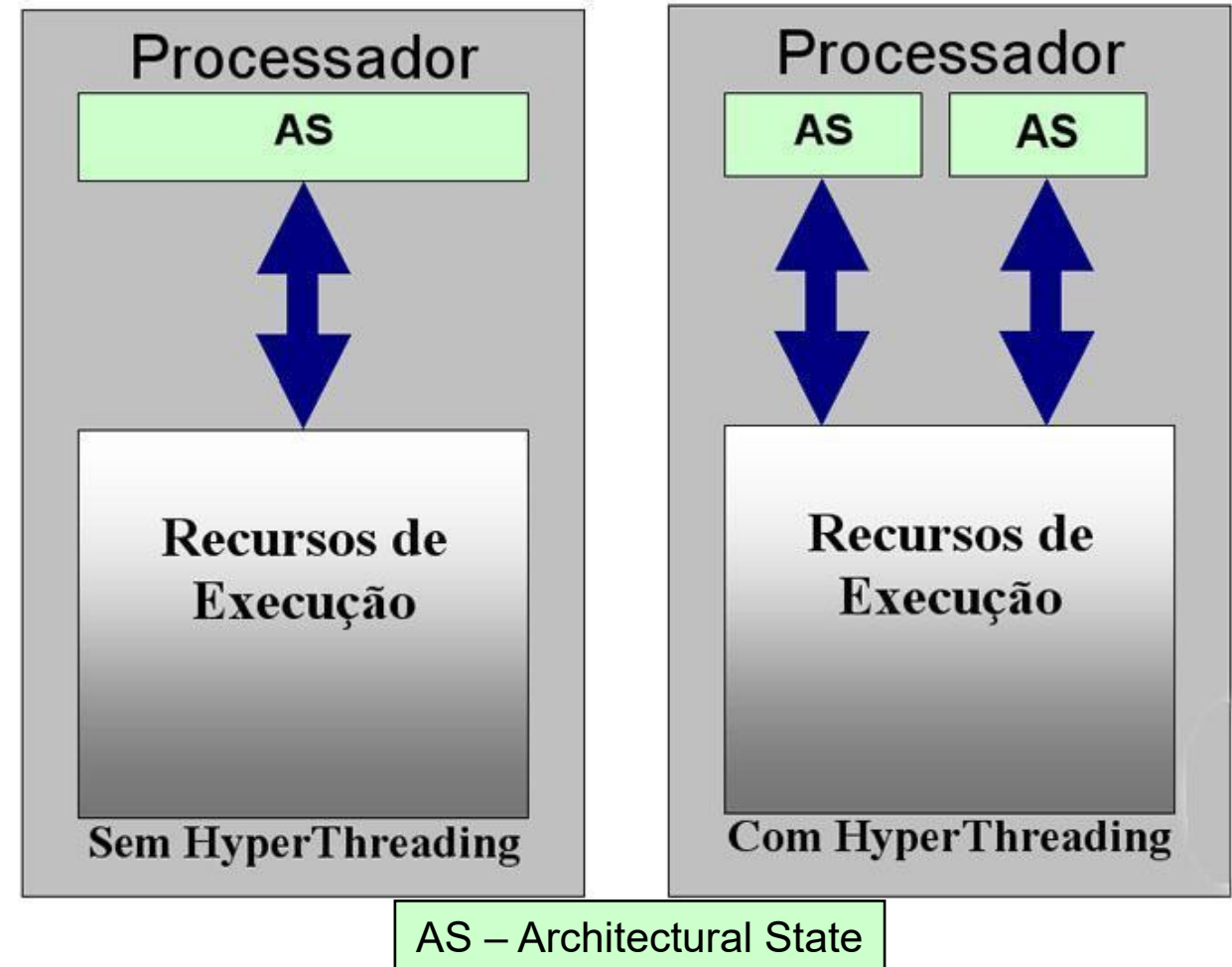
Objetivo

- Melhor utilização de recursos (cache e unidades de execução são compartilhadas entre threads)
- Ganho de desempenho (em média 20%)
- Baixo consumo de área ($< 5\%$ da área do chip)

HYPER-THREADING

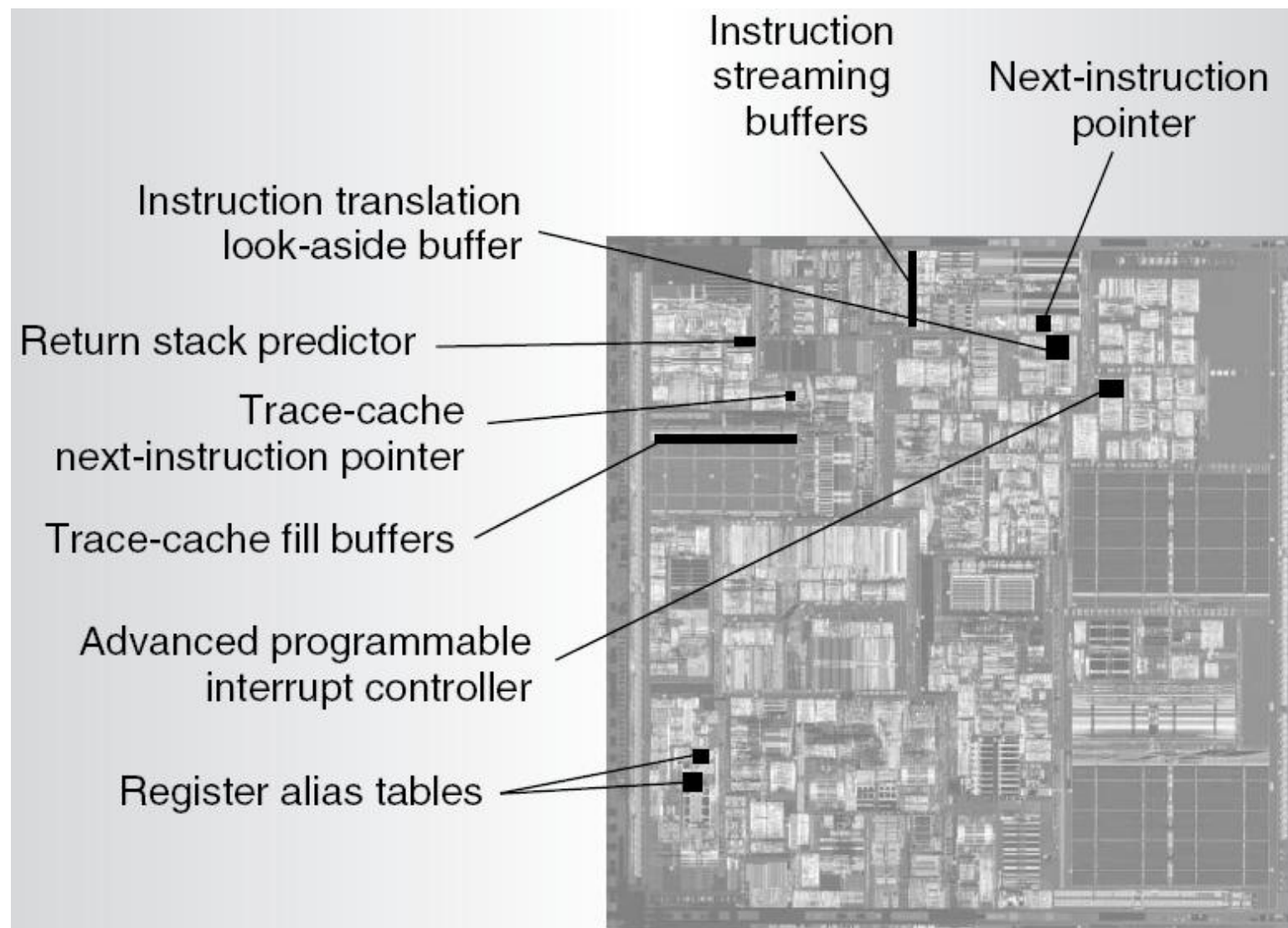
Componentes adicionados a um processador tradicional

- lógica de controle e duplicação do módulo referente ao contexto do processo em execução (pilha, registradores de controle, etc.)
- permitir concorrência na execução dos processos
- lógica de controle e duplicação do controlador de interrupções
- permitir a gerência concorrente de interrupções

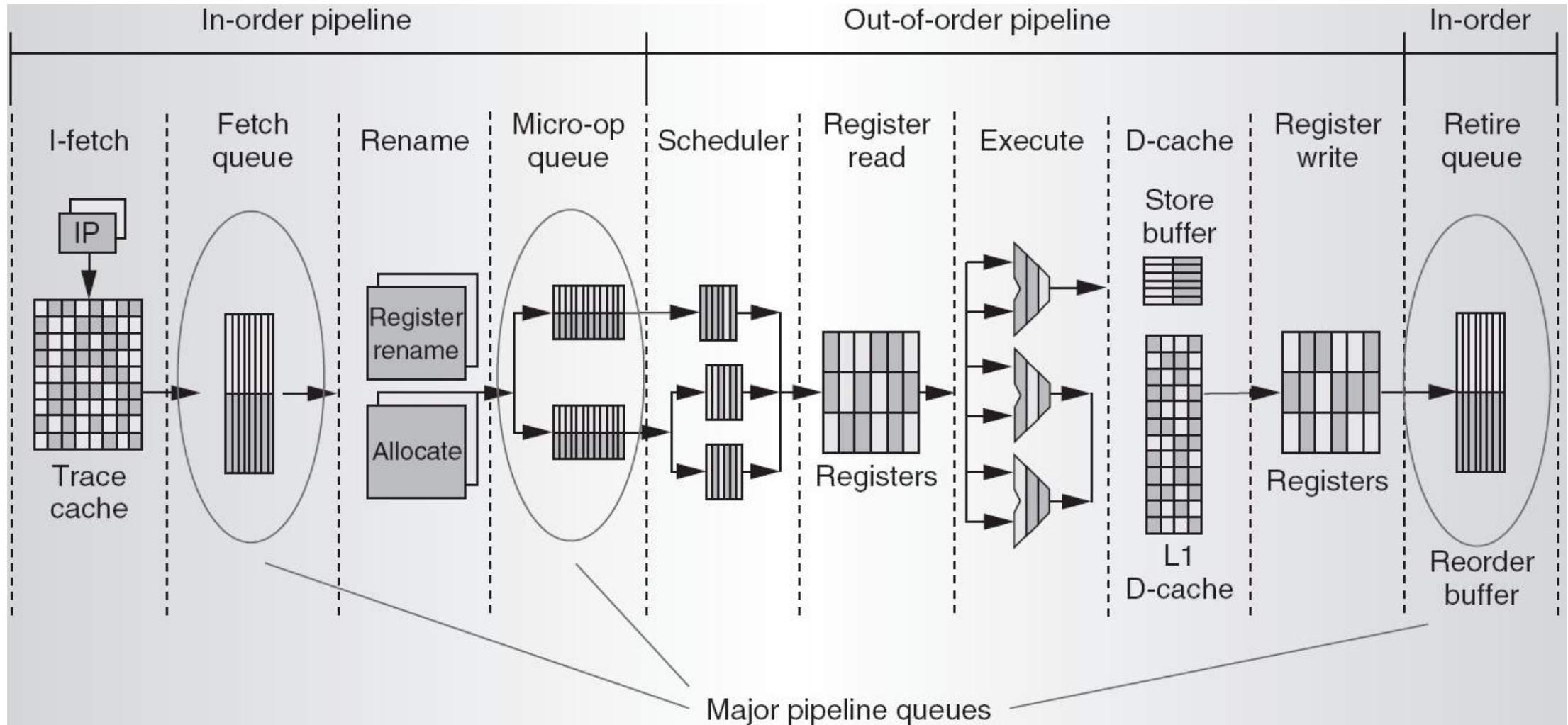


HYPER-THREADING

Partes replicadas
estão salientadas



HYPER-THREADING



As cores cinza claro e escuro representam duas threads usando os recursos do processador

HYPER-THREADING

Arquiteturas superescalares executam um único programa, ou thread, a partir do qual o processador tenta encontrar múltiplas instruções por ciclo para emitir. Quando não consegue, os slots não são utilizados e podem ocorrer na linha ou coluna (*stall*).

Os processadores multi-threads trocam (light switching) a thread em execução quando da ocorrência de um stall

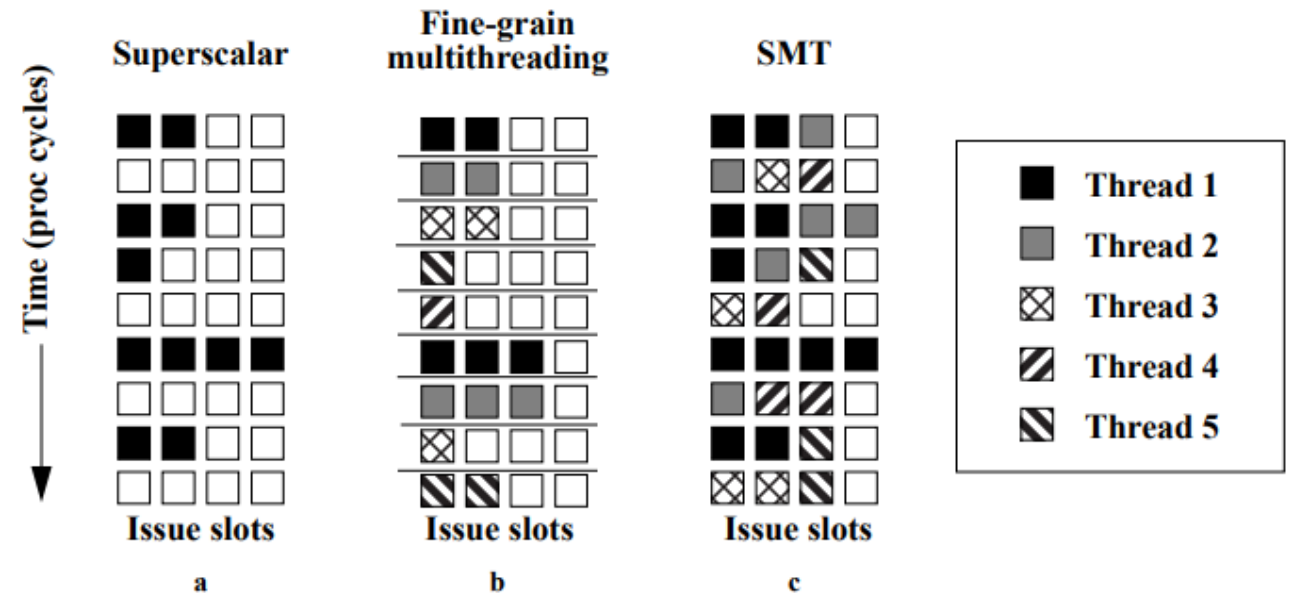
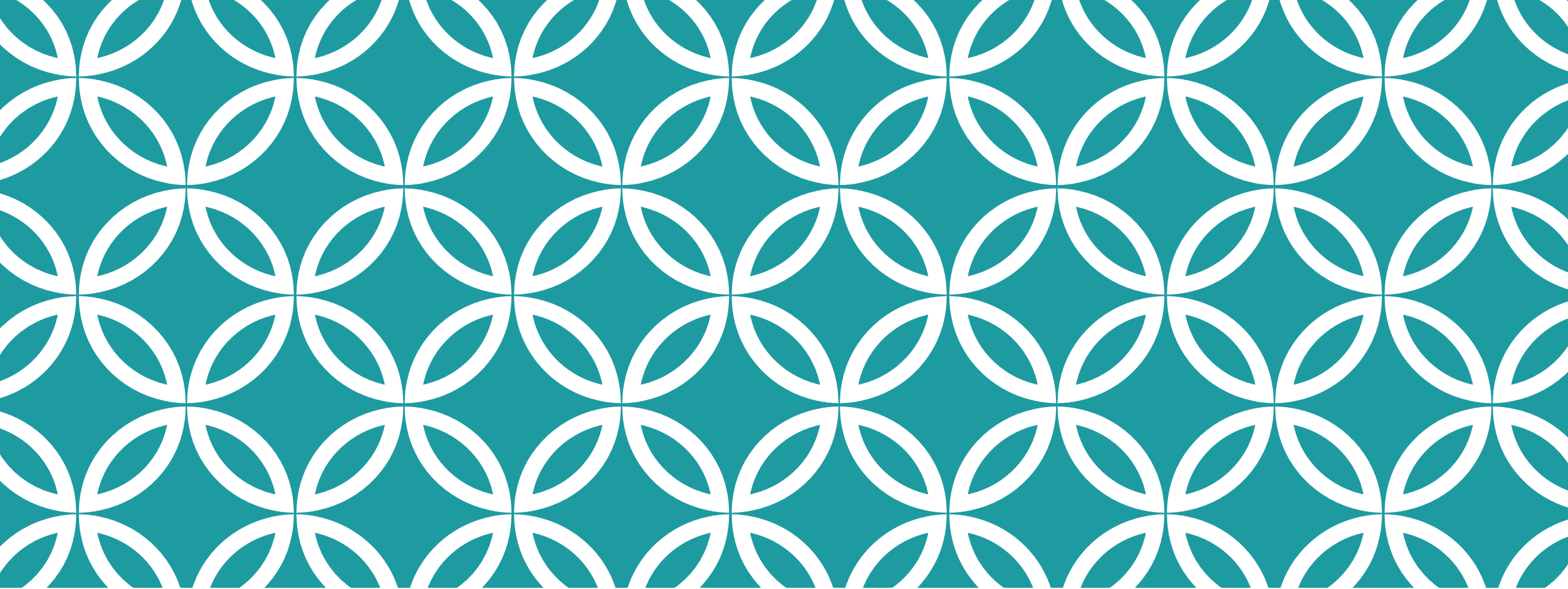


Figure 1: Comparison of issue slot (functional unit) partitioning in various architectures.

Eggers, S. J., Emer J. S., Levy, H. M., Lo, J. L., Stamm, R. L., Tullsen, D. M. Simultaneous Multithreading: A Platform for Next-Generation Processors. IEEE Micro, (Set/Out 1997), 12-19



ARQUITETURAS VETORIAIS

ARQUITETURAS VETORIAIS

Processador possui conjunto de instruções especiais para operações com vetor

Exemplo

VADD, VMUL ($V \times V \rightarrow V$)

VSUM, VMAX ($V \rightarrow S$)

VSQR, VSIN ($V \rightarrow V$)

Escalar
DO 100 I = 1,N
100 A(I) = B(I) + C(I)

Vetorial
A(1:N) = B(1:N) + C(1:N)

ULAs são replicadas e implementadas com pipelines aritméticos para acelerar execução das instruções vetoriais

Grandes bancos de registradores são utilizados para alimentar pipelines de forma eficiente (acabam sendo usados como caches)

ARQUITETURAS VETORIAIS

Processor (year)	Vector clock rate (MHz)	Vector registers	Elements per register (64-bit elements)	Vector arithmetic units	Vector load-store units	Lanes
Cray-1 (1976)	80	8	64	6: FP add, FP multiply, FP reciprocal, integer add, logical, shift	1	1
Cray X-MP (1983)	118	8	64	8: FP add, FP multiply, FP reciprocal, integer add, 2 logical, shift, population count/parity	2 loads 1 store	1
Cray Y-MP (1988)	166					
Cray-2 (1985)	244	8	64	5: FP add, FP multiply, FP reciprocal/sqrt, integer add/shift/population count, logical	1	1
Fujitsu VP100/VP200 (1982)	133	8–256	32–1024	3: FP or integer add/logical, multiply, divide	2	1 (VP100) 2 (VP200)
Hitachi S810/S820 (1983)	71	32	256	4: FP multiply-add, FP multiply/divide-add unit, 2 integer add/logical	3 loads 1 store	1 (S810) 2 (S820)
Convex C-1 (1985)	10	8	128	2: FP or integer multiply/divide, add/logical	1	1 (64 bit) 2 (32 bit)
NEC SX/2 (1985)	167	8 + 32	256	4: FP multiply/divide, FP add, integer add/logical, shift	1	4
Cray C90 (1991)	240	8	128	8: FP add, FP multiply, FP reciprocal, integer add, 2 logical, shift, population count/parity	2 loads 1 store	2
Cray T90 (1995)	460					
NEC SX/5 (1998)	312	8 + 64	512	4: FP or integer add/shift, multiply, divide, logical	1	16
Fujitsu VPP5000 (1999)	300	8–256	128–4096	3: FP or integer multiply, add/logical, divide	1 load 1 store	16
Cray SV1 (1998)	300	8	64 (MSP)	8: FP add, FP multiply, FP reciprocal, integer add, 2 logical, shift, population count/parity	1 load-store 1 load	2 8 (MSP)
SV1ex (2001)	500					
VMIPS (2001)	500	8	64	5: FP multiply, FP divide, FP add, integer add/shift, logical	1 load-store	1
NEC SX/6 (2001)	500	8 + 64	256	4: FP or integer add/shift, multiply, divide, logical	1	8
NEC SX/8 (2004)	2000	8 + 64	256	4: FP or integer add/shift, multiply, divide, logical	1	4
Cray X1 (2002)	800	32	64 256 (MSP)	3: FP or integer, add/logical, multiply/shift, divide/square root/logical	1 load 1 store	2 8 (MSP)
Cray XIE (2005)	1130					

ARQUITETURAS VETORIAIS

Cray 1 (1976)

- 8 MB de RAM
- 80 MHz
- 12 unidades funcionais com pipeline
- 8 registradores vetoriais cada um com 64 elementos de 64 bits
- 10 instruções vetoriais
- 3 tipos básicos
 - Endereços (24-bit)
 - Inteiros (64-bit)
 - Ponto flutuante (64-bit, 48-bit mantissa)



ARQUITETURAS VETORIAIS

G1 - Vector units

Vector (integer) Add: 3 stages

Vector Logical: 2 stages

Vector Shift Arquiteturas : 4 stages

G2 - Vector and scalar units

Floating Add: 6 stages

Floating Multiply: 7 stages

Floating Approximation: 14 stages

G3 - Scalar units

Integer Add: 3 stages

Logical: 1 stage

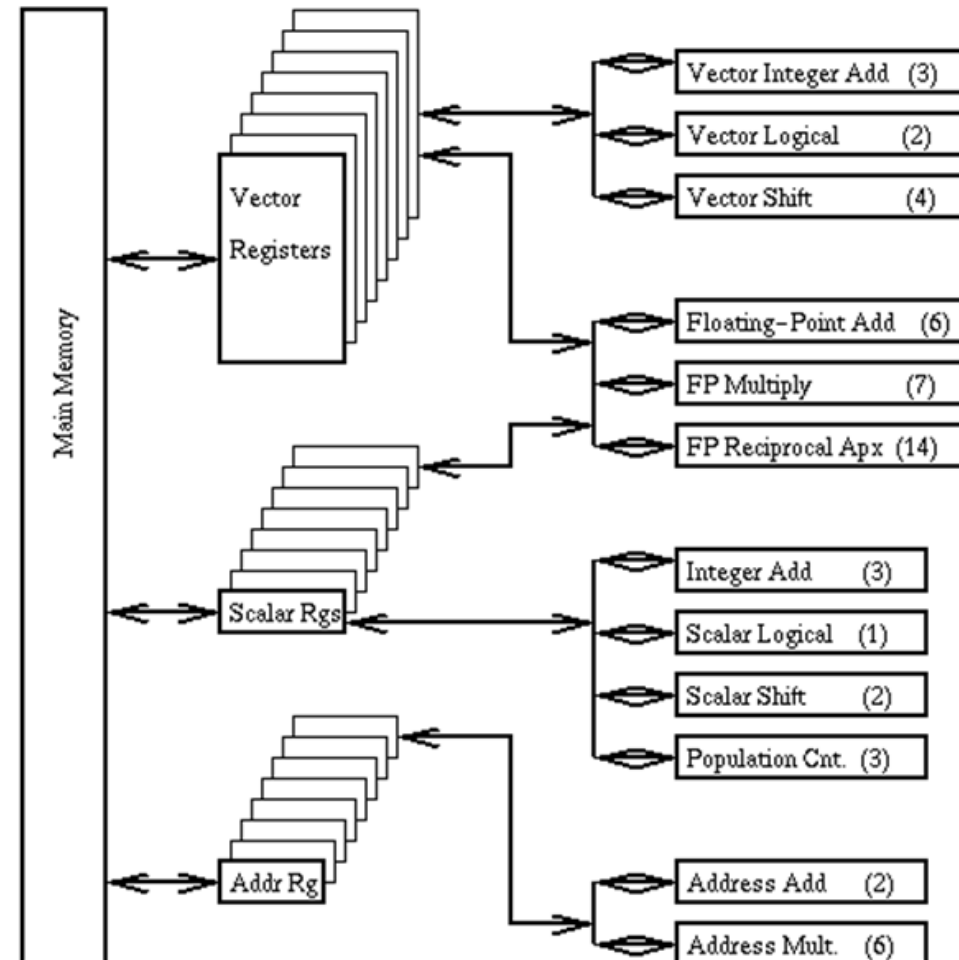
Shift: 2 stages

Scalar population count : 3 stages

G4 - Address units

Add: 2 stages

Multiply: 6 stages



ARQUITETURAS VETORIAIS

Cray T94

- Último processador vetorial da Cray lançado em 1995
- Adquirido pela UFRGS em 1998 (substituído em 2009 por um cluster da Sun)
- Processadores Vetoriais: 4
- Performance Teórica: 6 Gflops
- Pipelines Vetoriais: 2 por processador



ARQUITETURAS VETORIAIS

Exemplo da operação DAXPY em um processador MIPS escalar versus vetorial – $Y = a * X + Y$

```
for (i=0; i<64, i++) {  
    Y[i]=a*X[i]+Y[i];  
}
```

ARQUITETURAS VETORIAIS

Versão Escalar

- Assuma que os registradores Rx e Ry possuem os endereços de X e Y, respectivamente

	L.D F0, a ;	Carrega o escalar a
	DADDIU R4, Rx, #512 ;	Último endereço para carregar
LOOP:	L.D F2, 0(Rx) ;	Carrega X[i]
	MUL.D F2, F2, F0 ;	$a * X[i]$
	L.D F4, 0(Ry) ;	Carrega Y[i]
	ADD.D F4, F4, F2 ;	$a * X[i] + Y[i]$
	S.D F4, 0(Ry) ;	Armazena em Y[i]
	DADDIU Rx, Rx, #8 ;	Incrementa índice para X
	DADDIU Ry, Ry, #8 ;	Incrementa índice para Y
	DSUBU R20, R4, Rx ;	Calcula o limite
	BNEZ R20, Loop ;	Verifica se todo vetor está feito

ARQUITETURAS VETORIAIS

Versão Vetorial

L.D F0,a ;	Carrega o escalar a
LV V1,Rx ;	Carrega o vetor X
MULVS.D V2,V1,F0 ;	Multiplicação vetor - escalar
LV V3,Ry ;	Carrega o vetor Y
ADDVV.D V4,V2,V3 ;	Soma
SV Ry,V4 ;	Armazena o resultado

onde

ADDVV.D: soma dois vetores

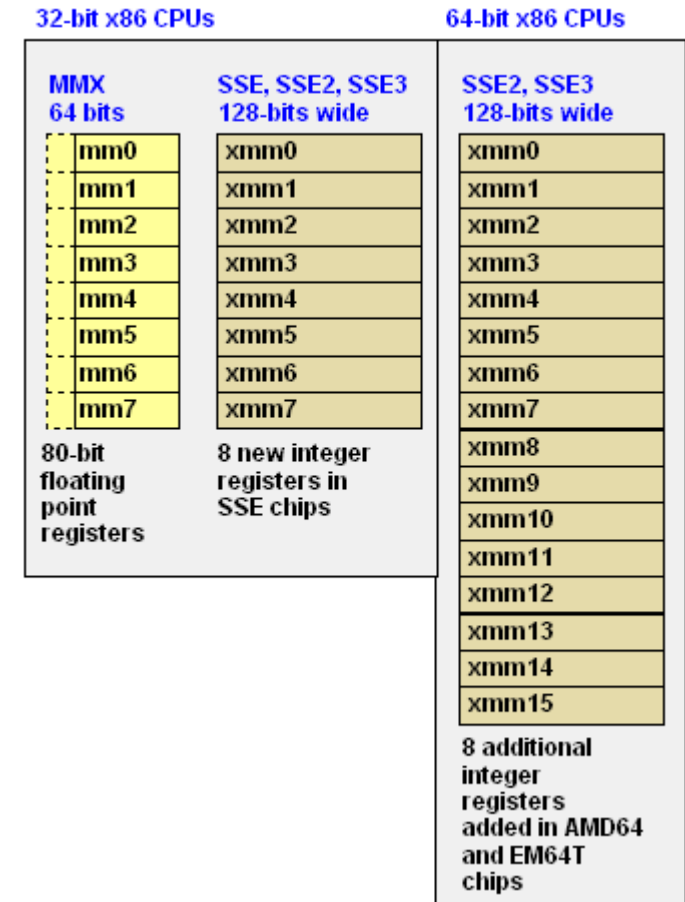
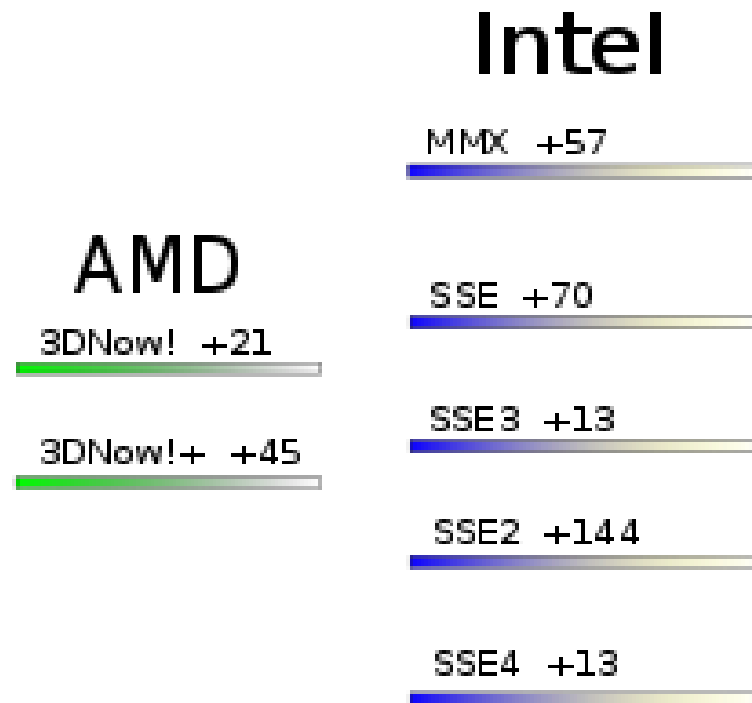
ADDVS.D: soma um vetor com um escalar

LV/SV: carrega / armazena vetor a partir de um endereço

ARQUITETURAS VETORIAIS

Como funciona SIMD na família x86?

- Registradores XMM* possuem 128 bits (16 bytes)



ARQUITETURAS VETORIAIS

Operação de produto escalar entre 2 vetores ($V1 = V1 * V2$)

- Vetores apontados por EBX (V2) e EAX (V1) e ECX é o índice

Versão x86

loop:

```
movups xmm0,[ebx+ecx];
```

Carrega 4 floats no reg xmm0

```
movups xmm1,[eax+ecx];
```

Carrega 4 floats no reg xmm1

```
mulps xmm0,xmm1;
```

Multiplica registradores vetoriais

```
movups [eax+ecx],xmm0;
```

Escreve de volta na memória

```
sub ecx,16;
```

Atualiza deslocamento (4 floats)

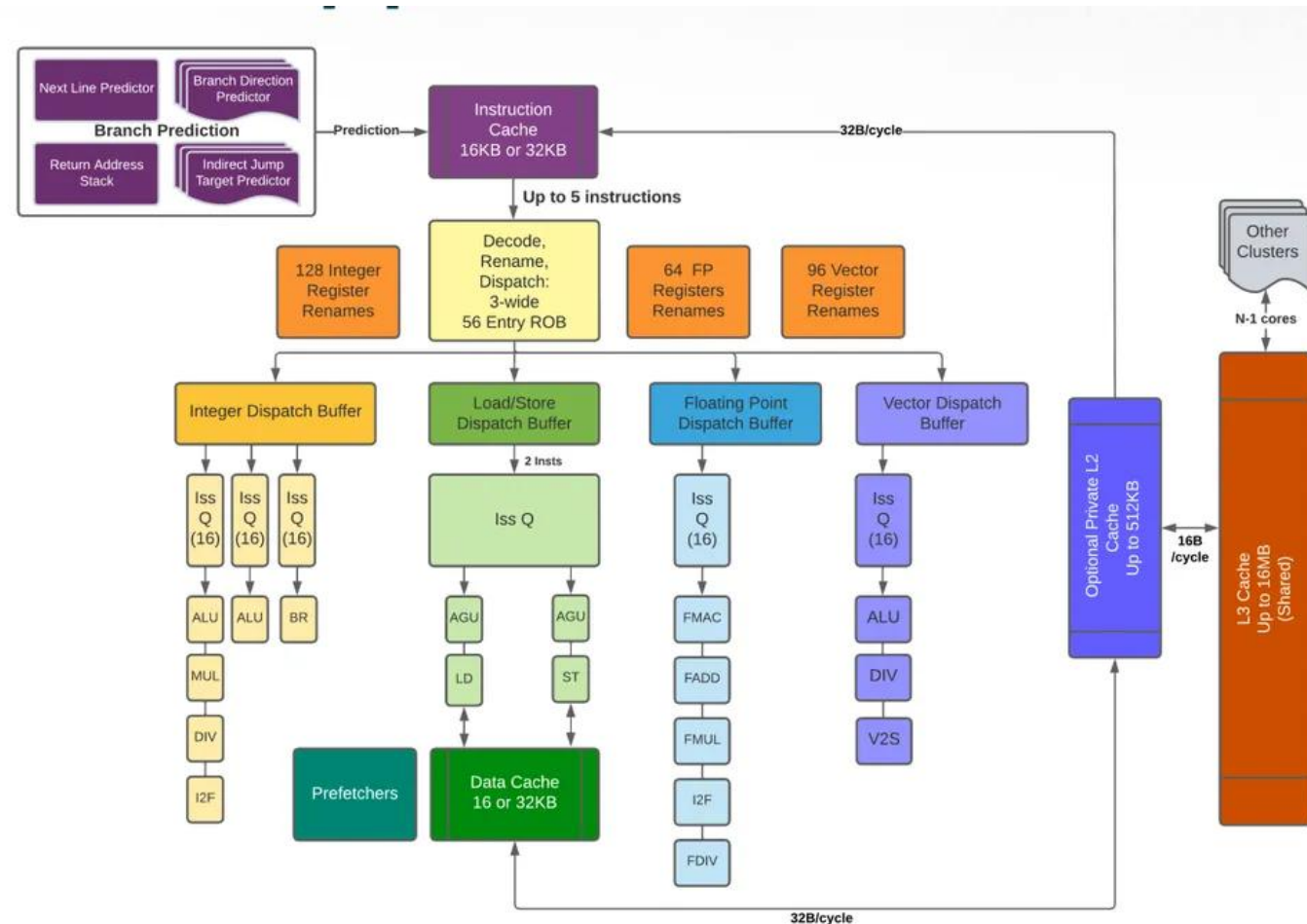
```
jnz loop
```

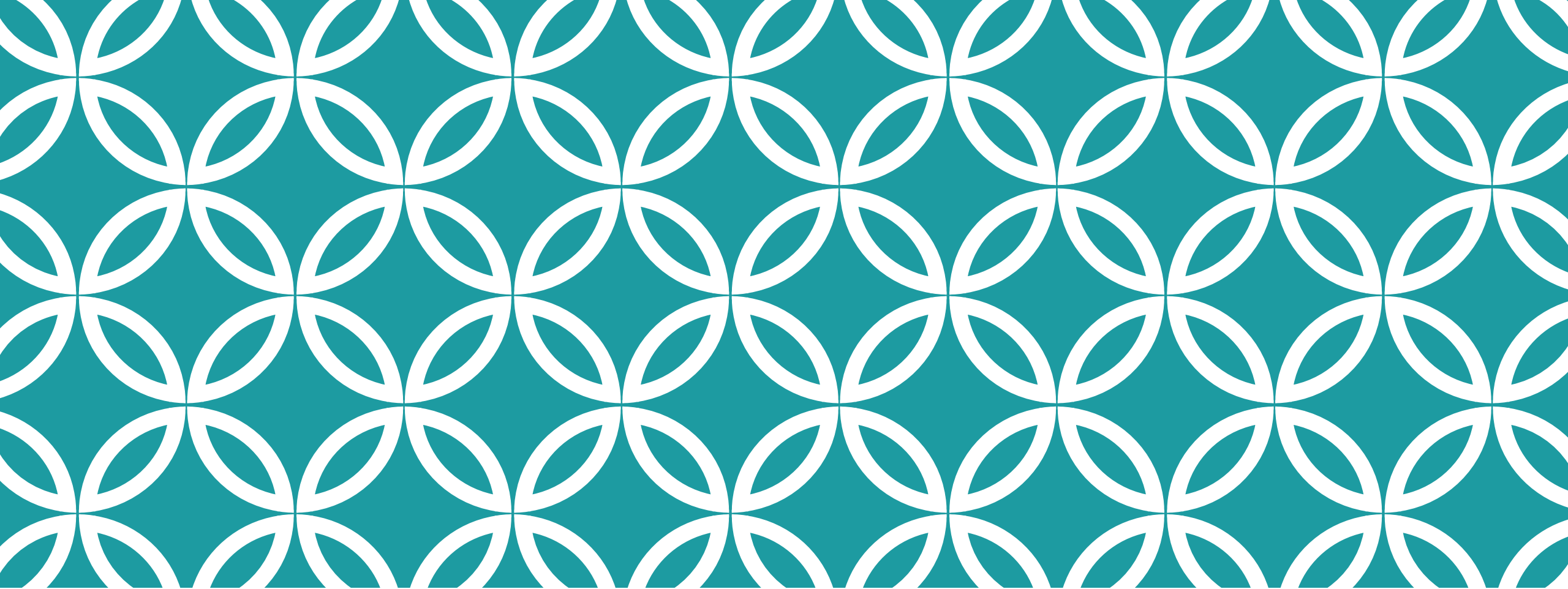
ARQUITETURAS VETORIAIS

O SiFive P600 pertence à família Performance Series da empresa SiFive, principal fornecedora comercial de processadores baseados em RISC-V. Ele é projetado para data centers, IA, HPC e aplicações embarcadas de alto desempenho, e implementa a RISC-V Vector Extension (RVV v1.0).

O P600 possui unidades vetoriais configuráveis, que permitem processar vários dados em paralelo com uma única instrução — exatamente como os antigos processadores Cray faziam, porém agora:

- o dentro de um núcleo RISC-V escalar,
- o com largura vetorial configurável por software (de 128 a 2048 bits), e
- o suporte a operadores de ponto flutuante e inteiros.





E UMA PERGUNTA...

POSCOMP 2008

32. Analise as seguintes afirmativas.

- I. Uma arquitetura multithreading executa simultaneamente o código de diversos fluxos de instruções (threads).
- II. Em uma arquitetura VLIW, o controle da execução das várias instruções por ciclo de máquina é feito pelo compilador.
- III. Uma arquitetura superescalar depende de uma boa taxa de acerto do mecanismo de predição de desvio para obter um bom desempenho.
- IV. Os processadores vetoriais são um tipo de arquitetura SIMD.
- V. Um processador dual-core é mais eficiente em termos de consumo de energia do que dois processadores *single-core* de mesma tecnologia.

A partir da análise, pode-se concluir que

- a) Apenas a afirmativa IV está correta.
- b) Apenas as afirmativas III e IV estão corretas.
- c) Apenas as afirmativas I, IV e V estão corretas.
- d) Apenas as afirmativas I, III e V estão corretas.
- e) Todas as afirmativas estão corretas.

POSCOMP 2008

32. Analise as seguintes afirmativas.

- I. Uma arquitetura multithreading executa simultaneamente o código de diversos fluxos de instruções (threads).
- II. Em uma arquitetura VLIW, o controle da execução das várias instruções por ciclo de máquina é feito pelo compilador.
- III. Uma arquitetura superescalar depende de uma boa taxa de acerto do mecanismo de predição de desvio para obter um bom desempenho.
- IV. Os processadores vetoriais são um tipo de arquitetura SIMD.
- V. Um processador dual-core é mais eficiente em termos de consumo de energia do que dois processadores *single-core* de mesma tecnologia.

A partir da análise, pode-se concluir que

- a) Apenas a afirmativa IV está correta.
- b) Apenas as afirmativas III e IV estão corretas.
- c) Apenas as afirmativas I, IV e V estão corretas.
- d) Apenas as afirmativas I, III e V estão corretas.
- e) Todas as afirmativas estão corretas.

POSCOMP 2011

29 Relacione a coluna da esquerda com a coluna da direita.

- | | |
|------------------------|--|
| (I) Multicore | (A) Múltiplos pipelines que operam em paralelo. |
| (II) Superpipeline | (B) Execução de instruções fora de ordem em um pipeline. |
| (III) Superescalar | (C) Pipelines com grande número de estágios. |
| (IV) Pipeline dinâmico | (D) Múltiplos processadores compartilhando um espaço de endereços. |
| (V) Multiprocessadores | (E) Múltiplos processadores em um único encapsulamento. |

Assinale a alternativa que contém a associação correta.

- a) I-B, II-A, III-C, IV-E, V-D.
- b) I-C, II-A, III-B, IV-D, V-E.
- c) I-D, II-E, III-B, IV-A, V-C.
- d) I-E, II-C, III-A, IV-B, V-D.
- e) I-E, II-C, III-A, IV-D, V-B.

POSCOMP 2011

29 Relacione a coluna da esquerda com a coluna da direita.

- | | |
|------------------------|--|
| (I) Multicore | (A) Múltiplos pipelines que operam em paralelo. |
| (II) Superpipeline | (B) Execução de instruções fora de ordem em um pipeline. |
| (III) Superescalar | (C) Pipelines com grande número de estágios. |
| (IV) Pipeline dinâmico | (D) Múltiplos processadores compartilhando um espaço de endereços. |
| (V) Multiprocessadores | (E) Múltiplos processadores em um único encapsulamento. |

Assinale a alternativa que contém a associação correta.

- a) I-B, II-A, III-C, IV-E, V-D.
- b) I-C, II-A, III-B, IV-D, V-E.
- c) I-D, II-E, III-B, IV-A, V-C.
- d) I-E, II-C, III-A, IV-B, V-D.**
- e) I-E, II-C, III-A, IV-D, V-B.

POSCOMP 2012

46 Com relação a processadores, considere as afirmativas a seguir.

- I. Arquiteturas Superescalares podem executar instruções concorrentemente em pipelines diferentes.
- II. O superpipeline permite a execução de duas tarefas em um único ciclo de clock do processador.
- III. Multiprocessadores simétricos compartilham a utilização da memória principal.
- IV. A utilização de uma memória cache L2 compartilhada em processadores multicore é vantajosa em threads que possuem alta localidade.

Assinale a alternativa correta.

- a) Somente as afirmativas I e II são corretas.
- b) Somente as afirmativas I e IV são corretas.
- c) Somente as afirmativas III e IV são corretas.
- d) Somente as afirmativas I, II e III são corretas.
- e) Somente as afirmativas II, III e IV são corretas.

POSCOMP 2012

46 Com relação a processadores, considere as afirmativas a seguir.

- I. Arquiteturas Superescalares podem executar instruções concorrentemente em pipelines diferentes.
- II. O superpipeline permite a execução de duas tarefas em um único ciclo de clock do processador.
- III. Multiprocessadores simétricos compartilham a utilização da memória principal.
- IV. A utilização de uma memória cache L2 compartilhada em processadores multicore é vantajosa em threads que possuem alta localidade.

Assinale a alternativa correta.

- a) Somente as afirmativas I e II são corretas.
- b) Somente as afirmativas I e IV são corretas.
- c) Somente as afirmativas III e IV são corretas.
- d) Somente as afirmativas I, II e III são corretas.**
- e) Somente as afirmativas II, III e IV são corretas.